Τμήμα Μηχανικών Η/Υ & Πληροφορικής · Πανεπιστήμιο Πατρών

Αρχές Γλωσσών Προγραμματισμού και Μεταφραστών

Οδηγός μελέτης από τις παλιές εξεταστικές

CEID_23Y206 · 4ο εξάμηνο · υποχρεωτικό · 6 ECTS

Επιμέλεια: Giorgos Kandilas

https://kdngiorgos.github.io/ceid-notes/

Βάση γνώσης CEID · έκδοση 2026.10 · έκδοση 1.0

DOI: 10.5281/zenodo.23434967

CC BY-NC-SA 4.0 για το κείμενο του οδηγού· τα θέματα ανήκουν στους διδάσκοντες του μαθήματος.

Περιεχόμενα

Πριν ξεκινήσεις 2

1 Το μάθημα σε μία σελίδα 2

2 Χάρτης των εξετάσεων 3

3 Δείκτες C και διαχείριση μνήμης 6

4 Μεταβίβαση παραμέτρων 11

5 Python: λίστες, λεξικά, pandas 19

6 Γραμματικές BNF και ιεραρχία Chomsky 24

7 Συντακτική ανάλυση LL(1) 28

8 Λεξική ανάλυση: αυτόματα και κανονικές εκφράσεις 32

9 Συναρτησιακός: στρατηγικές αποτίμησης 36

10 Map, reduce και λογικός προγραμματισμός 40

11 Ασκήσεις εξάσκησης 45

Πηγές και ευχαριστίες 48

Πριν ξεκινήσεις

Αυτός ο οδηγός βασίζεται σε **12 εξεταστικές** (2020–2026) της βάσης γνώσης CEID, στο κείμενο του οδηγού του μαθήματος και σε ό,τι γράφτηκε από φοιτητές στο Discord. Η βάση έχει **δεκαέξι** εξετάσεις συνολικά· οι τέσσερις (2013-09, 2015-06, 2021-02, 2021-09) περιέχουν ένα μόνο θέμα η καθεμία και δεν μετρούν στον χάρτη. Μερική μεταγραφή έχουν οι Ιούνιος 2021, Σεπτέμβριος 2022, Ιούνιος 2022, Ιούνιος 2023 (Παραλλαγή Α), Σεπτέμβριος 2023 (θέματα 4–6) και Ιούνιος 2026. Όπου μια μεταγραφή είναι μερική, το γράφω δίπλα στην εξεταστική στον τίτλο.

Σημείωση

Διαφορά από άλλα μαθήματα: οι λύσεις των παλιών θεμάτων είναι **σπάνιες** και στις περισσότερες περιπτώσεις **AI-unverified**. Όπου υπάρχει λύση, το σημειώνω. Για τις υπόλοιπες ασκήσεις δεν δίνεται κλειδί. Λύσε πρώτα μόνος σου και ελέγχει με τα βήματα της μεθόδου.

Τι σημαίνουν οι σημάνσεις

- ΕΠΙΣΗΜΗ ΛΥΣΗ — η λύση προέρχεται από τους διδάσκοντες (δεν υπάρχει καμία για αυτό το μάθημα).
- ΕΛΕΓΜΕΝΗ — εξήγηση ελεγμένη απέναντι σε επίσημη λύση.
- AI — λύση γραμμένη με τη βοήθεια AI χωρίς επίσημη λύση για σύγκριση.
- DISCORD — γνώμη φοιτητών από το Discord του CEID, με ημερομηνία. Χρήσιμη, αλλά ανεπίσημη. Η κοινοτική λύση φέρει την ίδια σήμανση, γιατί προέρχεται από φοιτητές.

1

Το μάθημα σε μία σελίδα

Τι είναι. Το μάθημα εξηγεί πώς σχεδιάζονται και υλοποιούνται οι γλώσσες προγραμματισμού και πώς δουλεύει ένας μεταφραστής. Η ύλη πηγαίνει από τη λεξική και τη συντακτική ανάλυση (κανονικές εκφράσεις, αυτόματα, γραμματικές BNF, LL(1)) μέχρι την εμβέλεια, τη διαχείριση μνήμης, τη μεταβίβαση παραμέτρων και τα παραδείγματα συναρτησιακού και λογικού προγραμματισμού (λ-λογισμός, αποτίμηση, map/reduce, Prolog). Στην ίδια εξέταση εμφανίζεται συχνά και η Python, με λίστες, λεξικά και pandas.

Διδάσκοντες 2025-26 (κατάλογος μαθημάτων): Ι. Γαροφαλάκης, Σπ. Σιούτας, Π. Χατζηδούκας, Ι. Βασιλόπουλος (ΕΔΙΠ).

Ώρες και πίστωση. 3 ώρες θεωρία, 1 ώρα φροντιστήριο, 2 ώρες εργαστήριο· 6 ECTS (κατάλογος 2025-26).

Πώς βαθμολογείται. Κατά το περίγραμμα, όπως το συνοψίζει ο κατάλογος, η βαθμολογία είναι **τελική γραπτή 100% συν εργασία** (flex/bison, Python). Κατά φοιτητές η πράξη είναι διαφορετική: DISCORD «περίπου 70% γραπτή + 30% flex/bison» (2025-04, όπως το καταγράφει ο οδηγός του μαθήματος). Ο οδηγός αναφέρει και την εναλλακτική «60% γραπτή + 20% flex/bison + 20% Python» για όποιον παραδίδει και το πρότζεκτ Python. Ο ακριβής τύπος δεν είναι επίσημα επιβεβαιωμένος· ρώτα τη γραμματεία πριν βασιστείς σε έναν από τους δύο.

Η εξέταση.

- **Μορφή:** όλες οι εξετάσεις της βάσης είναι ανοιχτού τύπου (`format: open`), με εξαίρεση ένα θέμα πολλαπλής επιλογής στο Σεπτέμβριο 2020.
- **Διάρκεια:** 150 λεπτά (2,5 ώρες), όπως τυπώνεται στις εξετάσεις Ιουνίου 2022, Ιουνίου 2024, Σεπτεμβρίου 2024, Ιουνίου 2025 (Παραλλαγή Α) και Ιουνίου 2026.

- **Σημειώσεις στην εξέταση:** Δεν βρέθηκαν πληροφορίες στο επίσημο υλικό. Ο οδηγός της βάσης (README) αναφέρει ότι η ερώτηση για τα επιτρεπόμενα βοηθήματα έχει τεθεί επανειλημμένα χωρίς σαφή καταγεγραμμένη απάντηση· ισχύει η επίσημη ανακοίνωση.
- **Βαρύτητα ανά θέμα:** τα ποσοστά που τυπώνονται πάνω στα θέματα αλλάζουν από εξέταση σε εξέταση και δεν αθροίζονται πάντα στο 100%. Ο Ιούνιος 2023 (Παραλλαγή Β) τυπώνει 40+20+20+20+10 = 110%.
- **Τυπικά θέματα:** ίχνη εκτέλεσης σε C (δείκτες), σε Pascal (εμβέλεια και μεταβίβαση παραμέτρων), σε Python (λίστες, λεξικά, pandas), αυτόματα και κανονικές εκφράσεις, γραμματικές BNF με LL(1) και ιεραρχία Chomsky, και αποτίμηση συναρτησιακών εκφράσεων (λ-λογισμός, Scheme, Python lambda, Prolog).

Τι λένε οι φοιτητές (κατά φοιτητές, ανώνυμα):

> *Κατά φοιτητές, είναι μάθημα «που παίρνεις εύκολα βαθμό» με σταθερή δομή θεμάτων, αρκεί να διαβάσεις τις επαναληπτικές διαφάνειες.* DISCORD *2024-09*

> *Ο Σεπτέμβριος θεωρείται από κάποιους «πιο δύσκολος από τον Ιούνιο», αν και όχι ομόφωνα.* DISCORD *2024-09*

> *Στο flex/bison η προφορική εξέταση ρωτά «σχεδόν τα πάντα» και απαιτεί εξήγηση του κώδικα.* DISCORD *2025-06*

> *Κάποιοι αναφέρουν βάση 3,5 για κάθε μέρος, αρκεί ο συνδυασμός γραπτής και πρότζεκτ να ξεπερνά το 5.* DISCORD *2026-06 (ακούσματα, χωρίς επιβεβαίωση)*

Προαπαιτούμενα. Δεν βρέθηκαν πληροφορίες (ο κατάλογος δεν ορίζει συνιστώμενα προαπαιτούμενα).

Σημείωση για την κάλυψη. Η εξέταση του Ιουνίου 2026 έχει μερική μεταγραφή (λείπει η αρχή του θέματος 5, το Python/pandas). Το θέμα αυτό δεν έχει ετικέτα στα δεδομένα του οδηγού και δεν μετράει στον χάρτη παρακάτω.

2 Χάρτης των εξετάσεων

Ο πίνακας μετρά σε πόσες από τις **12 εξετάσεις** (2020–2026) εμφανίζεται κάθε θεματική ενότητα, με βάση τις ετικέτες των θεμάτων. Οι εξετάσεις είναι αποσπάσματα ή μερικές μεταγραφές, οπότε οι αριθμοί είναι **κάτω όρια**, όχι ακριβείς αναλογίες. Η ενότητα «Εμβέλεια» μπαίνει στο κεφάλαιο της μεταβίβασης παραμέτρων, γιατί στα θέματα εμφανίζονται μαζί. Ο λ-λογισμός μπαίνει στο κεφάλαιο της αποτίμησης και η Prolog στο κεφάλαιο των συναρτησιακών, ως σημείωση.

Θεματική ενότητα	Εξετάσεις (από 12)	Τελευταία εξέταση	Κεφ.
Δείκτες C και διαχείριση μνήμης	**10**	Ιούνιος 2026	1
Μεταβίβαση παραμέτρων	**9**	Ιούνιος 2026	2
Python: λίστες, λεξικά, pandas	**9**	Ιούνιος 2025 (Β) · και Ιούνιος 2026 χωρίς ετικέτα	3
Γραμματικές BNF και ιεραρχία Chomsky	**6**	Σεπτέμβριος 2024	4
Συντακτική ανάλυση LL(1)	**5**	Ιούνιος 2026	5
Λεξική ανάλυση: αυτόματα και κανονικές εκφράσεις	**5**	Ιούνιος 2026	6
Συναρτησιακός: στρατηγικές αποτίμησης	**4**	Σεπτέμβριος 2024	7
Συναρτησιακός: map, reduce, lambda	**4**	Ιούνιος 2025 (Β)	8
Εμβέλεια και περιβάλλοντα αναφοράς	2	Σεπτέμβριος 2022	2
λ-λογισμός	1	Σεπτέμβριος 2022	7
Λογικός προγραμματισμός (Prolog)	1	Ιούνιος 2026	8 (σημείωση)

Σημείωση

Τι να μελετήσεις πρώτα. Ξεκίνα από τους **δείκτες C** και τη **μεταβίβαση παραμέτρων**, που εμφανίζονται σχεδόν σε κάθε εξέταση και αντιστοιχούν σε ίχνη εκτέλεσης που μπορείς να μάθεις να κάνεις με σταθερή μέθοδο. Μετά έλεγξε την **Python** (λίστες, λεξικά, pandas) και τις **γραμματικές BNF** με LL(1). Οι αυτόματα και η αποτίμηση συναρτήσεων είναι μικρότερα αλλά «σίγουρα» ερωτήματα. Η Prolog εμφανίστηκε μόνο στην τελευταία εξέταση, οπότε δεν είναι ακόμη σταθερή ενότητα.

Χρονολόγιο: πόσες ερωτήσεις ανά ενότητα και έτος (κενό = δεν εμφανίστηκε)

Ενότητα	’13	’15	’20	’21	’22	’23	’24	’25	’26	Σ
Μεταβίβαση παραμέτρων				2	1	2	2	2	1	**10**
Δείκτες C και διαχείριση μνήμης					2	3	2	2	1	**10**
Python: λίστες, λεξικά, pandas			1		2	2	2	2		**9**
Γραμματικές BNF και ιεραρχία Chomsky		1		2	1	2	2			**8**
Λεξική ανάλυση: αυτόματα και κανονικές εκφράσεις	1			1			1	2	1	**6**
Συντακτική ανάλυση LL(1)			2	1				2	1	**6**
Εμβέλεια και περιβάλλοντα αναφοράς			3		1					**4**
Συναρτησιακός: στρατηγικές αποτίμησης					1	2	1			**4**
Συναρτησιακός: map, reduce, lambda					1		1	2		**4**
λ-λογισμός					1					**1**
Λογικός προγραμματισμός (Prolog)									1	**1**

Τύποι ερωτήσεων (63 ερωτήσεις, 55 από το 2021 και μετά)

Τύπος	Όλα	%	Από 2021
Σχεδίαση / εκτέλεση αλγορίθμου	22	35%	22
Υπολογισμός	19	30%	19
Επεξήγηση / θεωρία	16	25%	14
Πολλαπλή επιλογή	6	10%	0

Τα δύο παραπάνω σχήματα παράγονται αυτόματα από την ταξινόμηση των ερωτήσεων· μία ερώτηση με υποερωτήματα μετράει μία φορά.

3

Δείκτες C και διαχείριση μνήμης

Μέθοδος

(μέθοδος με δικά μας λόγια)
Η ιδέα. Ένας δείκτης κρατάει μια διεύθυνση. Όταν γράφεις `p + k`, η C προχωρά κατά `k` **στοιχεία** του τύπου, όχι κατά `k` byte. Το `p[k]` είναι το ίδιο με το `*(p + k)`. Ο αριθμός των byte ενός τύπου ορίζει την απόσταση σε μνήμη, αλλά στις ασκήσεις τη δουλεύεις πάντα σε στοιχεία.
Η διαδικασία. Ζωγράφισε τον πίνακα ως σειρά κουτιών με τις αρχικές τιμές, σημείωσε με βέλος πού δείχνει το `p` και εκτέλεσε τις γραμμές με τη σειρά, ενημερώνοντας τον πίνακα μετά από κάθε γραμμή. Το `p++` και το `p--` αλλάζουν τη θέση του δείκτη, όχι την τιμή που δείχνει. Για τους τελεστές `+=`, `-=` και `(*(p-2))++` ακολούθησε την εκφώνηση: στην ανάλυση που γράφουμε εδώ, η παλιά τιμή χρησιμοποιείται στην πράξη και η αύξηση γίνεται μετά.
Έλεγχος. Πριν την τελική απάντηση, γράψε το άθροισμα ή το άθροισμα των αλλαγών (π.χ. «αφαιρέθηκε 1 από το data[1]»): τα λάθη συνήθως είναι στις πράξεις και όχι στη λογική.

Παράδειγμα: δείκτες με αριθμητική και ανάθεση (Ιούνιος 2026, θέμα 3, μερική μεταγραφή)

Θέμα εξέτασης

Ποιες θα είναι οι τελικές τιμές του πίνακα data που θα τυπώσει το παρακάτω πρόγραμμα. Αναφέρετε και εξηγήστε τους υπολογισμούς σας.

```
void main() {
  int *p, data[] = {6, 12, 18, 24};
  p = &data[2];
  *(p - 2) = *p / 3 + 1;
  p--;
  *(p + 2) = *p * 2 - 1;
  *p += p[1] / 3;
  *(p + 1) = *(p + 2) - *p;
  printf("%d %d %d %d\n", data[0], data[1], data[2], data[3]);
}
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Το αρχείο της εξέτασης δεν έχει λύση, οπότε η παρακάτω είναι δική μας ανάλυση.
Αρχικά data = {6, 12, 18, 24} και ο `p` δείχνει στο `data[2]` (18).

1. `*(p - 2) = *p / 3 + 1;` → data[0] = 18/3 + 1 = 7. Πίνακας: {7, 12, 18, 24}.
2. `p--;` → ο `p` δείχνει στο `data[1]`.
3. `*(p + 2) = *p * 2 - 1;` → data[3] = 12·2 − 1 = 23. Πίνακας: {7, 12, 18, 23}.
4. `*p += p[1] / 3;` → data[1] = 12 + 18/3 = 18. Πίνακας: {7, 18, 18, 23}.
5. `*(p + 1) = *(p + 2) - *p;` → data[2] = 23 − 18 = 5. Πίνακας: {7, 18, 5, 23}.

Εκτύπωση: `7 18 5 23`.
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να μετράς το `p - 2` ή το `p + 2` σε byte. Σε `int` ο δείκτης προχωρά κατά ένα στοιχείο για κάθε μονάδα. Επίσης, μετά το `p--` όλα τα `p[...]` αλλάζουν βάση· αν ξεχάσεις να ενημερώσεις τη θέση του `p`, θα πάρεις λάθος στοιχείο.

Παράδειγμα: δείκτες, τελεστές αύξησης και σύνθετη ανάθεση (Ιούνιος 2025, Παραλλαγή Α, θέμα 4)

Θέμα εξέτασης

Ποιες θα είναι οι τελικές τιμές του πίνακα data που θα τυπώσει το παρακάτω πρόγραμμα; Αναφέρετε και εξηγήστε τους υπολογισμούς σας.

```
1  void main() {
2  int *p, data[] = {2, 4, 6};
3  p = &data[1];
4  *(p+1) -= 1;
5  p = &p[1];
6  *(p-1) -= 1;
7  p += ((p-2))++;
8  printf("%d %d %d\n", data[0], data[1], data[2]);
9  }
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Το αρχείο λύσεων της εξέτασης (ai-unverified) γράφει `+= 1` στη γραμμή 4 και τελικές τιμές {3, 3, 9}. Η εκφώνηση γράφει `-= 1`, οπότε ξαναυπολογίζουμε:
Αρχικά data = {2, 4, 6}, ο `p` δείχνει στο `data[1]`.

- Γραμμή 4: `*(p+1) -= 1` → data[2] = 6 − 1 = 5.
- Γραμμή 5: `p = &p[1]` → ο `p` δείχνει στο `data[2]`.
- Γραμμή 6: `*(p-1) -= 1` → data[1] = 4 − 1 = 3.
- Γραμμή 7: `*p += (*(p-2))++` → η παλιά τιμή του `data[0]` (2) προστίθεται στο `data[2]`: data[2] = 5 + 2 = 7· μετά αυξάνεται το `data[0]` σε 3.

Τελικές τιμές: **{3, 3, 7}**. Η διόρθωση συμφωνεί με την προειδοποίηση του αρχείου λύσεων.
Λύση από AI — ελέγξτε την.

Σημείωση

Η παγίδα είναι η διαφορά ανάμεσα σε `-=` και `+=` στο ίδιο κελί: αν αντιγράψεις τη λύση χωρίς να ελέγξεις την εκφώνηση, παίρνεις λάθος αποτέλεσμα. Γράφε τη σειρά των ενημερώσεων μία φορά για κάθε γραμμή.

Παράδειγμα: ίχνος εκτέλεσης με τελεστές αύξησης και δείκτες (Ιούνιος 2023, Παραλλαγή Β, θέμα 2)

Θέμα εξέτασης

Δίνεται το παρακάτω τμήμα προγράμματος σε γλώσσα C:

Γραμμή	Κώδικας
1	`int main()`
2	`{`
3	`int data[6] = {10, 20, 30, 40, 50, 60};`
4	`int *p;`
5	`p = &data[2];`
6	`*(p-1) = (*(p-2))--;`
7	`*p += 1;`
8	`*++p = 44;`
9	`*p++ = 33;`
10	`p++;`
11	`*p = (*p > 50)? *p-1: *p+1;`
12	`for (int i=0; i<6; i++) printf("%d\n", data[i]);`
13	`return 0;`
14	`}`

1. Εξηγήστε γραμμή-γραμμή, για τις γραμμές 3-11, τους ορισμούς και τη λειτουργία του παραπάνω προγράμματος.
2. Τι θα τυπωθεί με την εκτέλεση του προγράμματος, δηλαδή ποιες θα είναι οι τελικές τιμές του πίνακα data;

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Το αρχείο λύσεων (ai-unverified) δίνει μόνο τις τελικές τιμές. Τις ξαναυπολογίσαμε για το κείμενο αυτό και συμφωνούν.
Αρχικά {10, 20, 30, 40, 50, 60}, ο p στο `data[2]`.

- Γραμμή 6: το `*(p-2)` είναι το `data[0]` (10) και το `--` επιστρέφει 10 και μειώνει το `data[0]` σε 9· το `data[1]` παίρνει 10.
- Γραμμή 7: `data[2]` = 30 + 1 = 31.
- Γραμμή 8: `*++p` → ο p γίνεται `data[3]`, τιμή 44.
- Γραμμή 9: `*p++ = 33` → `data[3]` = 33, ο p γίνεται `data[4]`.
- Γραμμή 10: `p++` → ο p γίνεται `data[5]`.
- Γραμμή 11: `data[5]` = 60 > 50, άρα `60 - 1` = 59.

Τελικές τιμές: **{9, 10, 31, 33, 50, 59}**.
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να ξεχνάς ότι το *p++ διαβάζει ή γράφει στη **παλιά** θέση και μετά προχωράει. Επίσης, το *++p προχωράει πρώτα. Γράψε κάθε ενημέρωση του δείκτη σε ξεχωριστή γραμμή πριν συνεχίσεις.

Ακόμη λυμένα θέματα

Επιπλέον θέματα της ίδιας ενότητας. Οι λύσεις είναι γραμμένες με τη βοήθεια AI και σημαίνονται· ελέγξτε τες πριν βασιστείς σε αυτές.

3.4.1 Σεπτέμβριος 2024, θέμα 3

Θέμα εξέτασης

Δίνεται το παρακάτω τμήμα προγράμματος σε γλώσσα C:

```
| 1 | void main(){ |
| 2 |     int data1[4] = {4, 3, 2, 1}; |
| 3 |     int data2[4] = {10, 20, 30, 40}; |
| 4 |     int *p1, *p2, a = 1, b = 2, c = 3; |
| 5 |     p1 = &data1[3]; |
| 6 |     p2 = &data2[*p1]; |
| 7 |     a = *(p2+1) + 1; |
| 8 |     p1 -= 2; |
| 9 |     b = p1[1] + *(p2-1); |
[…]
10      *p2 /= 5;
11      c += *p1 + *p2 - 2;
12      printf("a=%d b=%d c=%d\n", a, b, c);
13 }
```

(a) Εξηγήστε συνοπτικά και γραμμή-γραμμή, για τις γραμμές 5-11, τη λειτουργία του παραπάνω προγράμματος.

(b) Τι θα τυπωθεί με την εκτέλεση του προγράμματος, δηλ ποιες θα είναι οι τελικές τιμές των a, b, c;

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή. Το αρχείο της εξέτασης δεν έχει λύση· η παρακάτω γράφτηκε με τη βοήθεια AI και ελέγχθηκε με πρόγραμμα.
Αρχικά `data1` = {4, 3, 2, 1}, `data2` = {10, 20, 30, 40}, a = 1, b = 2, c = 3.

1. Γραμμή 5: `p1 = &data1[3]` → ο `p1` δείχνει στο 1.
2. Γραμμή 6: `p2 = &data2[*p1]` = `&data2[1]` → ο `p2` δείχνει στο 20.
3. Γραμμή 7: `a = *(p2+1) + 1` = `data2[2] + 1` = 31.
4. Γραμμή 8: `p1 -= 2` → ο `p1` δείχνει στο `data1[1]` (τιμή 3).
5. Γραμμή 9: `b = p1[1] + *(p2-1)` = `data1[2] + data2[0]` = 2 + 10 = 12.
6. Γραμμή 10: `*p2 /= 5` → `data2[1]` = 20 / 5 = 4.
7. Γραμμή 11: `c += *p1 + *p2 - 2` = 3 + 4 − 2 = 5, άρα c = 3 + 5 = 8.

Τελικές τιμές: **a = 31, b = 12, c = 8**. Εκτύπωση: `a=31 b=12 c=8`.
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να υπολογίζεις το `p1[1]` με τον `p1` πριν από τη γραμμή 8. Κάθε γραμμή που αλλάζει έναν δείκτη, τον διαβάζει από εκεί και πέρα στη νέα του θέση.

3.4.2 Ιούνιος 2022, θέμα 3 (β)

Θέμα εξέτασης

Δίνεται το παρακάτω τμήμα κώδικα σε C++:

```
int main()
{
    int data[6] = {1, 2, 3, 4, 5, 6};
    int *p, *q;
    int a, b, c;
    p = data;
    q = p + 2;
    a = *p + *q;
    b = q[2] + data[2];
    *(q - 1) = 0;
    p++;
    q++;
    c = *p - *q + data[3];
    cout << a << " " << b << " " << c << "\n";
    return 0;
}
```

1. Εξηγήστε γραμμή-γραμμή τους αριθμούς και τη λειτουργία του παραπάνω προγράμματος.
2. Τι θα τυπωθεί με την εκτέλεση του προγράμματος;

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: μέτρια (η φωτογραφία της εξέτασης ήταν δυσανάγνωστη στον κώδικα C++· η λύση βασίζεται στο κείμενο της μεταγραφής).
Αρχικά data = {1, 2, 3, 4, 5, 6}.

- `p = data` → ο p δείχνει στο `data[0]`· `q = p + 2` → ο q δείχνει στο `data[2]`.
- `a = *p + *q` = 1 + 3 = **4**.
- `b = q[2] + data[2]` = `data[4]` + `data[2]` = 5 + 3 = **8**.
- `*(q - 1) = 0` → `data[1]` = 0 (πίνακας 1, 0, 3, 4, 5, 6).
- `p++` → ο p δείχνει στο `data[1]`· `q++` → ο q δείχνει στο `data[3]`.
- `c = *p - *q + data[3]` = 0 − 4 + 4 = **0**.

Έξοδος: **4 8 0**.
Λύση από AI — ελέγξτε την.

4

Μεταβίβαση παραμέτρων

Μέθοδος

(μέθοδος με δικά μας λόγια· τα τέσσερα μοντέλα παρουσιάζονται και στα διαφανειά, βλ. πηγές)
Η εμβέλεια. Με **στατική** εμβέλεια, ένα όνομα που δεν ανήκει στο τοπικό περιβάλλον αναζητείται στο περιβάλλον όπου ορίστηκε η διαδικασία (το «περιβάλλον αναφοράς» της). Οι μη-τοπικές μεταβλητές βρίσκονται έτσι, όχι με βάση ποιος κάλεσε ποιον. Γράψε τα περιβάλλοντα αναφοράς κάθε τμήματος πριν απαντήσεις σε οτιδήποτε άλλο.
Τα τέσσερα μοντέλα. - *Κλήση με τιμή:* η παράμετρος παίρνει αντίγραφο της τιμής. Αλλαγές μέσα στη διαδικασία δεν φαίνονται έξω. - *Κλήση με αναφορά:* η παράμετρος είναι ψευδώνυμο της μεταβλητής που δόθηκε. - *Κλήση με τιμή-αποτέλεσμα:* αντίγραφο στην είσοδο, και η τελική τιμή γράφεται πίσω στο όρισμα στο τέλος. - *Κλήση με όνομα:* η έκφραση του ορίσματος αποτιμάται ξανά σε κάθε χρήση της παραμέτρου, στο περιβάλλον της κλήσης. Οι αλλαγές σε μεταβλητές που εμπλέκονται στο όρισμα μπορεί να φανούν στις επόμενες αναφορές.
Η διαδικασία. Γράψε τον πίνακα των τιμών των μεταβλητών του κύριου προγράμματος πριν την κλήση, και για κάθε τρόπο μετάβασης τα βήματα της διαδικασίας με τη σειρά. Τα αποτελέσματα των τεσσάρων τρόπων συνήθως διαφέρουν. Αν η εκφώνηση ζητά «τιμή-αποτέλεσμα» χωρίς να λέει ξεκάθαρα ποιο μοντέλο, ακολούθησε τον ορισμό της εκφώνησης.

Παράδειγμα: κλήση με τιμή σε Pascal με καθολικές μεταβλητές (Ιούνιος 2023, Παραλλαγή Β, θέμα 3)

Θέμα εξέτασης

Δίνεται το παρακάτω πρόγραμμα σε μια γλώσσα τύπου Pascal που χρησιμοποιεί το *στατικό κανόνα εμβέλειας*:

```
program MAIN;
var n: integer;
var A: array [1..5] of integer;
procedure F(k, b: integer);
begin
  writeln(k);
  n:= n + 1;
  writeln(k);
  k:= k + 4;
  writeln(n);
  b:= b + 5;
end;
BEGIN
  A[1]:= 10; A[2]:= 20; A[3]:= 30; A[4]:= 40; A[5]:= 50;
  n:= 1;
  F(n, a[n+2]);
  writeln(n);
  writeln(A[1], ' ', A[2], ' ', A[3], ' ', A[4], ' ', A[5]);
END.
```

(b) Ποιες τιμές θα τυπωθούν με τις εντολές writeln, έφοσον η διαδικασία F καλείται με τις παρα-

κάτω περιπτώσεις τρόπου μεταβίβασης παραμέτρων; Εξηγήστε σύντομα τις διαδικασίες υπολογισμού σε όλες τις περιπτώσεις.

1. Κλήση με τιμή (call by value).

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Η λύση του αρχείου 2023-06-a (ίδιος κώδικας, ai-unverified) καλύπτει μόνο την κλήση με τιμή. Το αρχείο συζητά και κλήση με αναφορά, που δεν ισχύει εδώ, γιατί η κεφαλίδα είναι `F(k, b: integer)`.
Με κλήση με τιμή, το k και το b είναι αντίγραφα. Η n είναι καθολική και αλλάζει κανονικά:

1. `writeln(k)` → **1**.
2. `n := n + 1` → n = 2.
3. `writeln(k)` → **1** (το k δεν επηρεάζεται από την n).
4. `k := k + 4` → τοπικό k = 5, χωρίς επίδραση έξω.
5. `writeln(n)` → **2**.
6. `b := b + 5` → τοπικό b, χωρίς επίδραση στο `A[3]`.
7. Μετά την κλήση: `writeln(n)` → **2**, και `writeln(...)` των πίνακα → **10 20 30 40 50**.

Έξοδος: 1, 1, 2, 2, «10 20 30 40 50».
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να αλλάζεις το k και να περιμένεις ότι η αλλαγή φτάνει στην n ή αντίστροφα. Μόνο οι καθολικές μεταβλητές (εδώ η n) αλλάζουν έξω από τη διαδικασία. Για τα ορίσματα του τύπου `A[n+2]` γράψε πρώτα την τιμή του δείκτη και μετά την κλήση.

Παράδειγμα: εμβέλεια και τέσσερις τρόποι μεταβίβασης (Ιούνιος 2025, Παραλλαγή Α, θέμα 3)

Θέμα εξέτασης

Δίνεται το παρακάτω πρόγραμμα σε μια γλώσσα τύπου Pascal στην οποία ισχύει ο *στατικός κανόνας εμβέλειας*:

```
program MAIN;
  var a: array[1..2] of integer; i, j: integer;

  procedure P(x: integer; y: integer);
  begin
    a[i] := a[i] + x;
    i := 2;
    x := x + 1;
    j := 1;
    y := y + x;
  end;

begin
  a[1] := 3; a[2] := 5;
```

```
  i := 1; j := 2;
  P(a[i], a[j]);
  writeln('output: ', a[1], ' ', a[2]);
end.
```

(a) Ποια είναι τα περιβάλλοντα αναφοράς όλων των τμημάτων του προγράμματος;

Λύση

Χωρίς κλειδί. Το θέμα δεν έχει λύση στο αρχείο λύσεων (η σχετική λύση του παλιού αρχείου αφορά άλλο πρόγραμμα). Η μέθοδος: για κάθε τρόπο μετάβασης γράψε πρώτα ποια τιμή παίρνει το `i` και το `j` **κατά τη στιγμή** κάθε χρήσης. Στην **κλήση με αναφορά** το `x` είναι ψευδώνυμο του `a[i]` της κλήσης, οπότε το `a[i]` μέσα στη διαδικασία εξαρτάται από το `i` κάθε στιγμή. Στην **κλήση με όνομα** το `a[i]` αποτιμάται ξανά σε κάθε χρήση, άρα η γραμμή `a[i] := a[i] + x` δίνει άλλο αποτέλεσμα από τις άλλες. Κράτα τον πίνακα των τιμών του `a[1]`, `a[2]` και γράψε κάθε αλλαγή.

Σημείωση

Η κλήση με όνομα είναι το πιο συχνό λάθος σε θέματα τύπου «τι τυπώνει». Θυμίσου ότι το όρισμα δεν είναι τιμή αλλά **έκφραση** που ξαναποτιμάται στο περιβάλλον της κλήσης. Επίσης, σε μία παραλλαγή (Ιούνιος 2025, Παραλλαγή Β) η τρίτη περίπτωση είναι γραμμένη «κλήση με τιμή» κατά λάθος (`[sic]`)· ο όρος του μαθήματος είναι «τιμή-αποτέλεσμα».

Παράδειγμα: εμβέλεια με ένθετες διαδικασίες (Σεπτέμβριος 2020, θέμα 2, ερώτημα Ι)

Θέμα εξέτασης

Ι. Θεωρώντας ότι ισχύει ο στατικός κανόνας εμβέλειας: a. Ποια είναι τα Περιβάλλοντα Αναφοράς (Τοπικά, Μη-τοπικά, Καθολικά) όλων των τμημάτων του προγράμματος;

```
program MAIN;
  procedure X;
    a: integer;
    procedure Y;
    begin
      a:= a*2;
      write("In Y:", a)
    end;
    procedure Z;
      a: integer;
    begin
      a:= 7;
      Y;
      write("In Z:", a)
    end;
  begin
    a:= 5;
    Z
    write("In X:", a)
```

```
  end;
BEGIN
  a:= 1;
  X;
END.
```

Λύση

Χωρίς κλειδί. Η μέθοδος: για κάθε διαδικασία γράψε τα **τοπικά** της ονόματα, τα **μη-τοπικά** (ορισμένα σε εξωτερικό περιβάλλον) και τα **καθολικά** (του κυρίως προγράμματος). Στη στατική εμβέλεια η Y είναι ορισμένη μέσα στο X, οπότε το a που χρησιμοποιεί η Y είναι το a του X και όχι το a του Z. Αυτό ακριβώς ελέγχει το ερώτημα. Το ερώτημα (II) του ίδιου θέματος αφορά τη δυναμική εμβέλεια και τα αποτελέσματα αλλάζουν.

Σημείωση

Συχνό λάθος: να βρίσκεις το «σωστό» a ψάχνοντας τη διαδικασία που κάλεσε τη Y. Με στατική εμβέλεια ψάχνεις το περιβάλλον όπου **ορίστηκε** η Y.

Ακόμη λυμένα θέματα

4.4.1 Σεπτέμβριος 2024, θέμα 2

Θέμα εξέτασης

Δίνεται το παρακάτω πρόγραμμα σε μια γλώσσα τύπου Pascal που χρησιμοποιεί το στατικό κανόνα εμβέλειας:

```
program MAIN;
var i, c: integer; a: array[1..2] of integer;
procedure F(x, y: integer);
  var d: integer;
  procedure G(a: integer);
  begin
    a := a + d;
  end;
begin
  d := -1;
  x := x - 1;
  G(i);
  G(y);
  c := x * y;
end;
BEGIN
  a[1] := 5;  a[2] := 4;  i:=2;
  F(a[i], a[i]);
  writeln('a[1]=', a[1]);  writeln('a[2]=', a[2]); writeln('c=', c);
END.
```

(a) Ποια είναι τα περιβάλλοντα αναφοράς (τοπικά, μη-τοπικά, καθολικά) όλων των τμημάτων του

προγράμματος;

(b) Ποιες τιμές (V1, V2, V3) θα τυπωθούν με την εντολή writeln, εφόσον οι διαδικασίες F και G καλούνται με τις παρακάτω περιπτώσεις τρόπου μεταβίβασης παραμέτρων; Εξηγήστε σύντομα τις διαδικασίες υπολογισμού σε όλες τις περιπτώσεις.

1. Όλες οι παράμετροι μεταβιβάζονται με κλήση με τιμή (call by value).
2. Όλες οι παράμετροι μεταβιβάζονται με κλήση με αναφορά (call by reference).
3. Όλες οι παράμετροι μεταβιβάζονται με τιμή – αποτέλεσμα (call by value-result).
4. Όλες οι παράμετροι μεταβιβάζονται με κλήση με όνομα (call by name).

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή. Το αρχείο της εξέτασης δεν έχει λύση· ο υπολογισμός έγινε εδώ με τον ορισμό του μαθήματος (μη-τοπικά = δηλώσεις ενδιάμεσων τμημάτων, καθολικά = του κυρίως προγράμματος).
(a) Περιβάλλοντα: - MAIN: τοπικά {i, c, a, F}. - F: τοπικά {x, y, d, G}· καθολικά {i, c, a, F}. - G: τοπικά {a} (η παράμετρος, που κρύβει τον πίνακα `a` του MAIN)· μη-τοπικά {x, y, d, G}· καθολικά {i, c, F}.
(b) Αρχικά a[1] = 5, a[2] = 4, i = 2· κλήση `F(a[2], a[2])` με d = –1 μέσα στη F.

1. *Τιμή:* x = 4 και y = 4 (αντίγραφα), x := 3. Το `G(i)` και το `G(y)` αλλάζουν μόνο τα τοπικά αντίγραφα. c = 3·4 = 12. Έξοδος: a[1] = 5, a[2] = 4, c = 12.
2. *Αναφορά:* x και y είναι ψευδώνυμα του a[2]. x := x – 1 → a[2] = 3. Το `G(i)` κάνει i := 2 – 1 = 1. Το `G(y)` κάνει a[2] := 3 – 1 = 2. c = 2·2 = 4. Έξοδος: a[1] = 5, a[2] = 2, c = 4.
3. *Τιμή-αποτέλεσμα:* x = 4, y = 4, x := 3. Το `G(i)` δίνει τοπικό a = 2 – 1 = 1 και το `G(y)` δίνει τοπικό a = 4 – 1 = 3, που γράφεται πίσω στο y = 3. c = 3·3 = 9. Στην έξοδο της F, τα x και y γράφονται στο a[2] (η διεύθυνση καθορίστηκε στην κλήση): a[2] = 3. Έξοδος: a[1] = 5, a[2] = 3, c = 9.
4. *Όνομα:* x ≡ a[i] και y ≡ a[i] με νέα αποτίμηση σε κάθε χρήση. x := x – 1 → a[2] = 3. Το `G(i)` κάνει i := 2 – 1 = 1. Τώρα το `G(y)` αποτιμά a[i] = a[1] και κάνει a[1] := 5 – 1 = 4. c = a[1]·a[1] = 16. Έξοδος: a[1] = 4, a[2] = 3, c = 16.

Λύση από AI — ελέγξτε την.

Σημείωση

Η κλήση με όνομα δίνει άλλο αποτέλεσμα επειδή το `i` αλλάζει μέσα στη διαδικασία και το όρισμα `a[i]` ξαναγίνεται αποτίμηση με τη νέα τιμή του.

4.4.2 Ιούνιος 2026, θέμα 4

Θέμα εξέτασης

Δίνεται το παρακάτω πρόγραμμα σε μια γλώσσα τύπου Pascal στην οποία ισχύει ο *στατικός κανόνας εμβέλειας*:

```
program MAIN;
  var b: array[1..3] of integer; k, m: integer;

  procedure Q(u: integer; v: integer);
  begin
    b[1] := b[1] + u;
```

```
    b[2] := b[2] + v;
    m := 3;
    u := u + 1;
    b[3] := b[3] + u;
    v := v + u;
  end;

begin
  b[1] := 1; b[2] := 2; b[3] := 3;
  k := 1; m := 2;
  Q(b[m], b[k]);
  writeln('output:', b[1], ' ', b[2], ' ', b[3]);
end
```

Μορφή εξόδου του προγράμματος: `output: b_1 b_2 b_3`

(a) Ποια είναι τα περιβάλλοντα αναφοράς όλων των τμημάτων του προγράμματος;

(b) Ποιες τιμές θα τυπωθούν στο τέλος για τα b[1], b[2] και b[3] στις παρακάτω περιπτώσεις μεταβίβασης παραμέτρων; Συμπληρώστε τον πίνακα (στην κόλα σας) και παρουσιάστε συνοπτικά τους υπολογισμούς σας.

1. Κλήση με τιμή (call by value)
2. Κλήση με αναφορά (call by reference)
3. Κλήση με τιμή-αποτέλεσμα (call by value-result)
4. Κλήση με όνομα (call by name)

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή.

(a) MAIN: τοπικά {b, k, m, Q}. Q: τοπικά {u, v}· καθολικά (του MAIN) {b, k, m, Q}.

(b) Αρχικά b = (1, 2, 3), k = 1, m = 2· κλήση `Q(b[2], b[1])`.

1. *Τιμή:* u = 2, v = 1 (αντίγραφα). b[1] = 1 + 2 = 3, b[2] = 2 + 1 = 3, m = 3, u = 3, b[3] = 3 + 3 = 6· το v γίνεται 4 τοπικά. Τελικά **(3, 3, 6)**.
2. *Αναφορά:* u ≡ b[2], v ≡ b[1]. b[1] = 1 + 2 = 3 (άρα v = 3)· b[2] = 2 + 3 = 5 (άρα u = 5)· m = 3· u := 6 (άρα b[2] = 6)· b[3] = 3 + 6 = 9· v := 3 + 6 = 9 (άρα b[1] = 9). Τελικά **(9, 6, 9)**.
3. *Τιμή-αποτέλεσμα:* όπως στην τιμή, και στην έξοδο γράφονται u → b[2] (= 3) και v → b[1] (= 4). Τελικά **(4, 3, 6)**.
4. *Όνομα:* u ≡ b[m], v ≡ b[k], με νέα αποτίμηση σε κάθε χρήση. b[1] = 1 + 2 = 3· b[2] = 2 + b[1] = 5· m = 3· u := u + 1 → b[3] = 4· b[3] = 4 + 4 = 8· v := v + u → b[1] = 3 + 8 = 11. Τελικά **(11, 5, 8)**.

Πίνακας αποτελεσμάτων: τιμή (3, 3, 6), αναφορά (9, 6, 9), τιμή-αποτέλεσμα (4, 3, 6), όνομα (11, 5, 8). Λύση από AI — ελέγξτε την.

Σημείωση

Η διαφορά τιμής και αναφοράς φαίνεται στο `b[1]` και `b[2]`: στην αναφορά τα u και v είναι το ίδιο κελί με τα `b[2]` και `b[1]`, οπότε κάθε αλλαγή τους γράφει στον πίνακα. Στην τιμή-αποτέλεσμα η διεύθυνση καθορίζεται στην κλήση, άρα η επιστροφή γράφει στο `b[2]` και `b[1]` και όχι στο `b[3]`.

4.4.3 Ιούνιος 2021, θέμα 3 (πρόγραμμα Α, ερώτημα (δ))

Θέμα εξέτασης

Δίνονται τα παρακάτω δύο προγράμματα μιας γλώσσας προγραμματισμού η οποία όσον αφορά το συντακτικό και τη γενική δομή των προγραμμάτων της είναι τύπου *Pascal*, ενώ υιοθετεί τις τεχνικές μεταβίβασης παραμέτρων καθώς και τη σημειογραφία και τη λειτουργικότητα δεικτών (pointers) της *C*. Τα προγράμματα εκτελούνται σε έναν compiler ο οποίος στην εντολή ανάθεσης προσδιορίζει πρώτα την *l-value* (αν χρειάζεται να προσδιοριστεί).
Πρόγραμμα Α

```
program MAIN;
    var n: integer;
        a: array [0..10] of integer;

    function P(x: integer): integer;
    begin
        write(x);
        n:= n + 2;
        write(x);
        x:= x + 8;
        P:= x;
    end;

BEGIN
    n:= 1;
    a[0]:= x; a[1]:= 100; a[2]:= 200;
    a[3]:= 300; a[4]:= 400; a[5]:= 500;
    a[6]:= 600;
    a[n+3]:= P(a[n+2]);
    write(a[1]);
    write(a[2]);
    write(a[3]);
    write(a[4]);
    write(a[5]);
END.
```

d. Ποια είναι η έξοδος των δύο προγραμμάτων κατά την εκτέλεσή τους; Εξηγείστε τις αλλαγές τιμών όλων των μεταβλητών κατά τη διαδικασία υπολογισμού.

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: μέτρια. Η εντολή `a[0]:= x` χρησιμοποιεί το `x`, που δεν είναι ορατό στο MAIN· είναι το σημασιολογικό λάθος του ερωτήματος (γ) και διαγράφεται πριν την εκτέλεση. Αρχικά n = 1, a[1..6] = 100, 200, 300, 400, 500, 600. Η ανάθεση `a[n+3]:=` ορίζει πρώτα τη θέση a[4], πριν η P αλλάξει το n.
Πρόγραμμα Α (κλήση με τιμή): το x είναι αντίγραφο του a[3] = 300. `write(x)` → 300· n = 3· `write(x)` → 300· x = 308, άρα P = 308 και a[4] = 308. Τελικές τιμές a[1..5] = 100, 200, 300, 308, 500. **Έξοδος: 300 300 100 200 300 308 500.**
Πρόγραμμα Β (δείκτης στο a[3]): `write(*x)` → 300· n = 3· `write(*x)` → 300· *x = 308, άρα a[3] = 308, P = 308 και a[4] = 308. Τελικές τιμές a[1..5] = 100, 200, 308, 308, 500. **Έξοδος: 300 300 100 200 308**

308 500.
Λύση από AI — ελέγξτε την.

Σημείωση

Η διαφορά των δύο προγραμμάτων φαίνεται στο a[3]: στο Α η αλλαγή του αντιγράφου δεν φτάνει στον πίνακα, στο Β φτάνει επειδή ο δείκτης δείχνει στο a[3].

5

Python: λίστες, λεξικά, pandas

Μέθοδος

(μέθοδος με δικά μας λόγια)

Βρόχοι και range. Το `range(a, b, s)` περιλαμβάνει το `a` και δεν περιλαμβάνει το `b`. Με αρνητικό βήμα μετράς προς τα κάτω. Το `break` σταματά όλο τον βρόχο, το `continue` μόνο την τρέχουσα επανάληψη. Όταν ένα θέμα έχει ένθετους βρόχους, γράψε ξεχωριστά τις τιμές της εξωτερικής και της εσωτερικής μεταβλητής σε κάθε βήμα.

Λίστες και λεξικά. Οι λίστες έχουν σειρά και αλλάζουν με `append`, `insert`, `remove`, `sort`. Τα λεξικά αντιστοιχίζουν κλειδιά σε τιμές και ο βρόχος `for k, v in d.items()` διατρέχει και τα δύο. Ένα πρόγραμμα με συνάρτηση που παίρνει λεξικό και επιστρέφει νέο λεξικό γράφεται πιο καθαρά με τις συναρτήσεις του ίδιου τύπου.

pandas. Το `DataFrame` είναι πίνακας με ονόματα στηλών. Η κάθετη συνένωση γίνεται με `pd.concat`, η συγχώνευση με `merge` σε κοινή στήλη, η διαγραφή στήλης με `drop(columns=...)` και το `iterrows()` επιστρέφει ζεύγη (δείκτης, γραμμή). Οι συναρτήσεις αυτές δεν τυπώνουν αυτόματα το αποτέλεσμα, γι' αυτό συχνά ξεχνιούνται.

Παράδειγμα: ίχνος βρόχου με break (Σεπτέμβριος 2022, θέμα 3α, μερική μεταγραφή)

Θέμα εξέτασης

(α) Ποιο το αποτέλεσμα της εκτέλεσης του τμήματος προγράμματος:

```
for x in range(12, 2, -2):
    print(x+2, end = ' ')
    if x == 4 :
        break
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Ακολουθούν τα βήματα της λύσης μας (δεν υπάρχει λύση στο αρχείο λύσεων).

Η `range(12, 2, -2)` δίνει 12, 10, 8, 6, 4 (το 2 δεν περιλαμβάνεται). Για κάθε τιμή τυπώνεται `x+2`, και στο `x == 4` σπάει ο βρόχος αφού τυπωθεί το 6.

Έξοδος: **14 12 10 8 6**

Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να θεωρείς ότι η `range` περιλαμβάνει το τελικό όριο. Εδώ το 2 δεν μπαίνει. Επίσης, το `break` σταματά **ολόκληρο** τον βρόχο που το περιέχει, με τις επόμενες τιμές του `range` να μένουν ανεκτέλεστες.

Παράδειγμα: πίνακας φοιτητών και βαθμών με pandas (Ιούνιος 2025, Παραλλαγή Β, θέμα 5)

Θέμα εξέτασης

Δίνεται η παρακάτω λίστα που περιέχει τα στοιχεία φοιτητών και τους βαθμούς τους σε δύο μαθήματα:

```
students = [
    {'Όνομα': 'Γιώργος', 'Μάθημα1': 7.5, 'Μάθημα2': 8.0},
    {'Όνομα': 'Μαρία', 'Μάθημα1': 9.0, 'Μάθημα2': 9.5},
    {'Όνομα': 'Κώστας', 'Μάθημα1': 6.0, 'Μάθημα2': 5.5},
    {'Όνομα': 'Άννα', 'Μάθημα1': 8.5, 'Μάθημα2': 9.0},
    {'Όνομα': 'Πέτρος', 'Μάθημα1': 5.0, 'Μάθημα2': 6.5},
]
```

(a) Χρησιμοποιώντας κατάλληλο βρόχο επανάληψης, εκτυπώστε για κάθε φοιτητή τα εξής στοιχεία: Όνομα — Μέσος Όρος Βαθμών.

Λύση

Χωρίς κλειδί. Η μέθοδος: διάτρεξε τη λίστα με βρόχο, υπολόγισε τον μέσο όρο των δύο βαθμών κάθε φοιτητή και εκτύπωσε το όνομα και τον μέσο. Στα επόμενα ερωτήματα (c), (d), (e) το ίδιο DataFrame παίρνει νέα στήλη, φιλτράρεται με συνθήκη και ταξινομείται σε φθίνουσα σειρά. Το τελικό ερώτημα (f) σβήνει τη στήλη `Μάθημα2`. Η εκφώνηση ζητά ρητά «κατάλληλο βρόχο», οπότε μη χρησιμοποιήσεις μόνο pandas στο (a).

Σημείωση

Συχνό λάθος: να μετράς τον μέσο όρο με λάθος παρονομαστή ή να ξεχνάς ότι η `drop` δεν αλλάζει το DataFrame αν δεν την αποθηκεύσεις ή δεν γράψεις `inplace=True`. Επίσης, ένα φίλτρο `df[df[...] >= 8]` επιστρέφει νέο DataFrame, όχι λίστα.

Ακόμη λυμένα θέματα

5.3.1 Σεπτέμβριος 2024, θέμα 5

Θέμα εξέτασης

Γράψτε ένα πρόγραμμα σε Python που να διαχειρίζεται μια λίστα αριθμών. Το πρόγραμμα θα πρέπει να χρησιμοποιεί τις κατάλληλες μεθόδους για την εκτέλεση των παρακάτω εργασιών: (α) Δημιουργήστε μια λίστα με τους εξής αριθμούς: - 5, 12, 8, 3, 25, 18, 22,25,30 (β) Πρόσθεσε στην αρχή της λίστας τον αριθμό 7 και στο τέλος της λίστας τον αριθμό 20. (γ) Διέγραψε τον αριθμό 8 από τη λίστα. (δ) Ταξινόμησε τη λίστα των αριθμών σε αύξουσα σειρά. (ε) Επίλεξε και τύπωσε τον δεύτερο και τον τέταρτο αριθμό από τη λίστα. (στ) Υπολόγισε και εμφάνισε το συνολικό πλήθος των αριθμών στη λίστα. (ζ) Επίλεξε και εμφάνισε τον αριθμό που βρίσκεται στη μέση της λίστας (αν το πλήθος είναι ζυγό, εμφάνισε τον μεσαίο αριθμό της κάτω μισής). (η) Αντικατάστησε τον τελευταίο αριθμό της λίστας με τον αριθμό 15.

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή. Το «-» μπροστά από το 5 στην εκφώνηση το διαβάζουμε ως σημείο λίστας, όχι ως πρόσημο. Δεν επιβεβαιώθηκε: το «- 5» της εκφώνησης μπορεί να είναι και αρνητικό πρόσημο (τότε το (ε) τυπώνει 3 12), και στο (ζ) το «μεσαίος της κάτω μισής» μπορεί να σημαίνει είτε το κάτω μεσαίο στοιχείο της λίστας (18, όπως εδώ) είτε το μεσαίο της πρώτης μισής λίστας (7).

```
nums = [5, 12, 8, 3, 25, 18, 22, 25, 30]      # (α)
nums.insert(0, 7)                             # (β) αρχή
nums.append(20)                               # (β) τέλος
nums.remove(8)                                # (γ) πρώτη εμφάνιση
nums.sort()                                   # (δ) αύξουσα
print(nums[1], nums[3])                       # (ε) δεύτερος και τέταρτος
print(len(nums))                              # (στ)
print(nums[(len(nums) - 1) // 2])          # (ζ) κάτω μεσαίος για ζυγό πλήθος
nums[-1] = 15                                 # (η)
```

Ροή τιμών: μετά το (β) `[7, 5, 12, 8, 3, 25, 18, 22, 25, 30, 20]`, μετά το (γ) `[7, 5, 12, 3, 25, 18, 22, 25, 30, 20]`, μετά το (δ) `[3, 5, 7, 12, 18, 20, 22, 25, 25, 30]`. Τυπώνονται: (ε) **5 12**, (στ) **10**, (ζ) **18**.
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να ξεχνάς ότι η `remove` διαγράφει μόνο την **πρώτη** εμφάνιση και ότι η `sort` αλλάζει τη λίστα στη θέση της. Για το (ζ) το πλήθος 10 δίνει δείκτη `(10 - 1) // 2 = 4`.

5.3.2 Ιούνιος 2026, θέμα 5

Θέμα εξέτασης

Ένα μετεωρολογικό κέντρο συλλέγει δεδομένα θερμοκρασίας από σταθμούς. Διαθέτει δύο λεξικά με μετρήσεις για δύο διαφορετικές ημέρες και ένα τρίτο λεξικό με τις γεωγραφικές πληροφορίες των σταθμών:

```
day1_data = {
    'Κωδικός': ['S1', 'S2', 'S3'],
    'Θερμοκρασία': [32.5, 29.0, 34.2]
}
day2_data = {
    'Κωδικός': ['S1', 'S2', 'S4'],
    'Θερμοκρασία': [31.0, 30.2, 28.5]
}
stations_info = {
    'Σταθμός_ID': ['S1', 'S2', 'S3', 'S4'],
    'Πόλη': ['Αθήνα', 'Πάτρα', 'Θεσσαλονίκη', 'Λάρισα'],
    'Υψόμετρο': [20, 15, 5, 75]
}
```

Γράψτε κώδικες Python, χρησιμοποιώντας τις κατάλληλες μεθόδους, ώστε να εκτελέσετε τα παρα-

κάτω βήματα:
(α) Να δημιουργήσετε τα DataFrames df_d1 και df_d2 από τα αντίστοιχα λεξικά των ημερών και να τα συνενώσετε κάθετα σε ένα ενιαίο DataFrame με όνομα df_all_measurements. (β) Να δημιουργήσετε το DataFrame df_stations από το λεξικό stations_info. Στη συνέχεια, να συγχωνεύσετε το df_all_measurements με το df_stations σε ένα τελικό DataFrame df_final. (γ) Στο df_final, να διαγράψετε οριστικά τη στήλη 'Υψόμετρο'. (δ) Χρησιμοποιώντας τη συνάρτηση iterrows(), να γράψετε ένα βρόχο επανάληψης που προσπελαύνει το df_final ανά γραμμή και εμφανίζει στην οθόνη για κάθε εγγραφή το μήνυμα: `Πόλη: <τιμή> | Θερμοκρασία: <τιμή>`.

Λύση
AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: μέτρια (ο κώδικας δεν δοκιμάστηκε με εκτέλεση· ακολουθεί το τυπικό API της pandas).

```
import pandas as pd

# (α)
df_d1 = pd.DataFrame(day1_data)
df_d2 = pd.DataFrame(day2_data)
df_all_measurements = pd.concat([df_d1, df_d2], ignore_index=True)  # κάθετη συνένωση

# (β)
df_stations = pd.DataFrame(stations_info)
df_final = pd.merge(df_all_measurements, df_stations,
                    left_on='Κωδικός', right_on='Σταθμός_ID', how='inner')

# (γ)
df_final.drop(columns='Υψόμετρο', inplace=True)

# (δ)
for index, row in df_final.iterrows():
    print(f"Πόλη: {row['Πόλη']} | Θερμοκρασία: {row['Θερμοκρασία']}")
```

Η `concat` με `ignore_index=True` αριθμεί ξανά τις γραμμές 0 έως 5. Όλες οι μετρήσεις έχουν αντίστοιχο σταθμό, άρα το `inner` δεν χάνει γραμμές. Έξοδος (με τη σειρά των γραμμών): Αθήνα 32.5, Πάτρα 29.0, Θεσσαλονίκη 34.2, Αθήνα 31.0, Πάτρα 30.2, Λάρισα 28.5.
Λύση από AI — ελέγξτε την.

5.3.3 Σεπτέμβριος 2022, θέμα 3 (γ)

Θέμα εξέτασης

(γ) Ποιο το αποτέλεσμα της εκτέλεσης του τμήματος προγράμματος:

```
i=5
while i > 0:
    i=i+1
    if i < 7: continue
    if i > 12: break
```

```
    print (i, end = ' ')
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή.
Ξεκινά i = 5. Στην πρώτη επανάληψη το i γίνεται 6, άρα `i < 7` και ο βρόχος συνεχίζει με `continue` χωρίς εκτύπωση. Από το i = 7 έως το 12 δεν ισχύει καμία από τις δύο συνθήκες και τυπώνεται κάθε τιμή. Για i = 13 ισχύει `i > 12` και γίνεται `break`.
Έξοδος: **7 8 9 10 11 12** (με κενό μετά από κάθε τιμή).
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: το `continue` δεν σταματά τον βρόχο αλλά πηγαίνει στον έλεγχο της συνθήκης του `while`. Το `break` σταματά ολόκληρο τον βρόχο.

6

Γραμματικές BNF και ιεραρχία Chomsky

Μέθοδος

(μέθοδος με δικά μας λόγια)
Το ρόλος της γραμματικής. Μια γραμματική BNF έχει μη τερματικά σύμβολα (στο `<...>`), τερματικά σύμβολα (τους χαρακτήρες της γλώσσας) και κανόνες παραγωγής. Η γλώσσα είναι το σύνολο των συμβολοσειρών από τερματικά σύμβολα που παράγονται από το αρχικό σύμβολο.
Ιεραρχία Chomsky. Ο τύπος 3 (κανονικές) έχει κανόνες μόνο με ένα μη τερματικό στη δεξιά πλευρά, στο τέλος (ή στην αρχή, για αριστερή γραμμικότητα). Ο τύπος 2 (χωρίς συμφραζόμενα) έχει ένα μη τερματικό στην αριστερή πλευρά και οτιδήποτε δεξιά. Αν υπάρχει μη τερματικό στη δεξιά πλευρά που δεν είναι τελευταίο, η γραμματική δεν είναι κανονική.
Παραγωγή και αναγνώριση. Για να δείξεις ότι μια συμβολοσειρά ανήκει στη γλώσσα, γράψε μια αλυσίδα παραγωγών από το αρχικό σύμβολο. Για να δείξεις ότι δεν ανήκει, εξήγησε γιατί κανένας συνδυασμός κανόνων δεν τη δίνει. Η αναγνώριση με στοίβα (από πάνω προς τα κάτω ή από κάτω προς τα πάνω) περιγράφεται στο κεφάλαιο 5.

Παράδειγμα: τύπος Chomsky γραμματικής με δεξιά γραμμικότητα (Ιούνιος 2023, Παραλλαγή Β, θέμα 1, δ)

Θέμα εξέτασης

```
<S> ::= int<κενό>id<A> | float<κενό>id<A>
<A> ::= ,<κενό>id<A> | ;
[…]
```

(d) Τι τύπου είναι η γραμματική στην ιεραρχία Chomsky; Αιτιολογήστε την απάντησή σας.

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Η λύση του αρχείου (ai-unverified) λέει **Τύπου 3 (κανονική)**, γιατί κάθε κανόνας έχει τη μορφή `A → w B` ή `A → w`, με `w` τερματικά και `B` μη τερματικό στο τέλος. Προσοχή: όπως είναι γραμμένη, η γραμματική έχει το μη τερματικό `<κενό>` στη **μέση** των κανόνων, οπότε ως γραμμένη δεν είναι ακόμη κανονική στον αυστηρό ορισμό. Αντικαθιστώντας το `<κενό>` με το τερματικό `_` προκύπτει κανονική γραμματική με την ίδια γλώσσα. Η απάντηση «τύπου 3» ισχύει λοιπόν για τη γλώσσα, με αυτή την αιτιολόγηση.
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να λες «κανονική» επειδή η γραμματική «μοιάζει απλή». Το κριτήριο είναι η θέση των μη τερματικών. Μια γραμματική με `<A> ::= <A><A> | 0 | 1` έχει δύο μη τερματικά στη δεξιά πλευρά και δεν είναι κανονική.

Παράδειγμα: ποιες συμβολοσειρές παράγονται (Σεπτέμβριος 2024, θέμα 1, β)

Θέμα εξέτασης

Σας δίνονται οι παρακάτω τρεις (3) απλές γραμματικές σε συμβολισμό BNF:

```
1. <A> ::= <A><A> | 0 | 1
2. <B> ::= 0<B> | 1<B> | 0
3. <C> ::= 0<C>0 | 1<C>1 | 0 | 1
```

(b) Αντιστοιχίστε τις ακολουθίες 10101, 011, 1, 000, 011100 στην/στις γραμματική/ές που τις παράγουν, ή σε καμία αν δεν μπορούν να παραχθούν.

Λύση

Χωρίς κλειδί. Η μέθοδος: για κάθε ακολουθία ρώτα πρώτα αν η γραμματική 2 παράγει οτιδήποτε με τερματικό τελευταίο (`0` ή `1`) χωρίς περιορισμό στο πλήθος. Η γραμματική 3 παράγει μόνο συμβολοσειρές που είναι **καθρέφτες**: το πρώτο και το τελευταίο ψηφίο ταιριάζουν και το κέντρο είναι ένα ψηφίο ή κενό. Για το τμήμα που δεν ταιριάζει, γράψε την πιο μικρή αλυσίδα παραγωγών που θα απέτυχε και εξήγησε γιατί. Η γραμματική 1 παράγει όλες τις μη κενές συμβολοσειρές, γιατί έχει `<A><A>`.

Σημείωση

Συχνό λάθος: να ξεχνάς ότι μια γραμματική με δύο ίδια μη τερματικά στη δεξιά πλευρά μπορεί να παράγει την ίδια συμβολοσειρά με πολλούς τρόπους. Αν ρωτήσουν «ποια συμβολοσειρά», δείξε μία αλυσίδα παραγωγών και μην γράψεις μόνο «ανήκει».

Ακόμη λυμένα θέματα

6.3.1 Φεβρουάριος 2021, θέμα 1 (Α)–(Γ)

Θέμα εξέτασης

1) Δίνεται η BNF γραμματική στο τέλος της εκφώνησης με επτά κανόνες R1-R7 και τερματικά σύμβολα 0 1

Α) Τί τύπου είναι η γραμματική στην ιεραρχία Chomsky. Αιτιολογήστε την απάντησή σας.
Β) Τι είδους συμβολοσειρές περιγράφει η παραπάνω γραμματική;
Γ) Μπορείτε να φτιάξετε μία κανονική έκφραση αντίστοιχη της γραμματικής και γιατί; Αν ναι παρουσιάστε την.
R1,R2: `<S>::= 0 <S>|1<A>` R3,R4: `<A>::= 0 <S> |1<B>` R5,R6, R7: `<B>::= 0 <B>|1< B>| $\epsilon$`

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή.
(Α) Κάθε κανόνας έχει ένα τερματικό ακολουθούμενο από το πολύ ένα μη τερματικό (ή το ε)· η γραμματική είναι δεξιά γραμμική, άρα **Τύπου 3 (κανονική)**.
(Β) Το S μένει στο S με `0` και πάει στο A με `1`. Το A με `0` γυρνά στο S (το μεμονωμένο `1` «ακυρώνεται»), με `1` πάει στο B. Το B δέχεται οτιδήποτε. Άρα η γλώσσα είναι **οι δυαδικές συμβολοσειρές που περιέχουν**

τουλάχιστον δύο διαδοχικά 1 (υποσυμβολοσειρά 11). Ελάχιστη: 11.
(Γ) Ναι, γιατί η γλώσσα είναι κανονική. Πριν από το πρώτο 11 κάθε 1 πρέπει να ακολουθείται από 0, άρα:

$$(0 \mid 10)^* \, 11 \, (0 \mid 1)^*$$

Έλεγχος με πρόγραμμα για όλες τις συμβολοσειρές μέχρι μήκος 12: η έκφραση ταυτίζεται με το «περιέχει 11».
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να λες «κανονική» επειδή η γραμματική μοιάζει απλή. Το κριτήριο είναι η θέση των μη τερματικών στους κανόνες.

6.3.2 Ιούνιος 2022, θέμα 1 (c)–(d)

Θέμα εξέτασης

Δίνεται η παρακάτω BNF γραμματική με αλφάβητο a b

$$< S >::= a < A > \mid b < B >$$

$$< A >::= a < S > \mid a$$

$$< B >::= b < S > \mid b$$

(c) Τι τύπου είναι η γραμματική στην ιεραρχία Chomsky; Αιτιολογήστε την απάντησή σας.
(d) Μπορείτε να φτιάξετε μια *κανονική έκφραση* αντιστοιχη της γραμματικής και γιατί; Αν ναι, παρουσιάστε την.

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή.
(c) Όλοι οι κανόνες έχουν τη μορφή τερματικό και το πολύ ένα μη τερματικό στο δεξί άκρο· η γραμματική είναι δεξιά γραμμική, άρα **Τύπου 3 (κανονική)**.
(d) Ναι: η γλώσσα είναι η $(aa \mid bb)^+$. Κάθε χαρακτήρας πρέπει να ακολουθείται από ίδιο χαρακτήρα, άρα τα γράμματα ζευγαρώνουν σε θέσεις (1–2, 3–4, …).
Λύση από AI — ελέγξτε την.

6.3.3 Ιούνιος 2026, θέμα 1 (α)–(β)

Θέμα εξέτασης

Δίνεται η παρακάτω απλή BNF γραμματική, με τερματικά σύμβολα τα: 0 1

$$< S >::= 0 < A > 0$$

$$< A >::= 1 < A > 1 \mid 1011$$

(α) Τι τύπου είναι η γραμματική στην ιεραρχία Chomsky και γιατί;
(β) Τι είδους συμβολοσειρές περιγράφει η παραπάνω γραμματική; Περιγράψτε τα χαρακτηριστικά των συμβολοσειρών αυτών. Δώστε σχετικά παραδείγματα, καθώς και τα δύο μικρότερα μέλη της γλώσσας.

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή.

(α) Ο κανόνας A → 1A1 έχει το μη τερματικό ανάμεσα σε τερματικά, άρα η γραμματική δεν είναι δεξιά ή αριστερά γραμμική. Είναι **Τύπου 2 (χωρίς συμφραζόμενα)** και όχι κανονική, γιατί για να ταιριάξει το πλήθος των 1 δεξιά και αριστερά χρειάζεται μέτρημα.

(β) Με n εφαρμογές του A → 1A1 και μία του A → 1011 παίρνουμε $A \Rightarrow^* 1^n\,1011\,1^n$. Άρα η γλώσσα είναι $\{\, 0\,1^n\,1011\,1^n\,0 \mid n \geq 0 \,\}$: αρχίζει και τελειώνει με 0, και στο κέντρο έχει τον πυρήνα 1011 με ίσο πλήθος 1 εκατέρωθεν. Παραδείγματα: 010110 (n = 0), 01101110 (n = 1), 0111011110 (n = 2). Τα δύο μικρότερα μέλη είναι τα 010110 (μήκος 6) και 01101110 (μήκος 8).

Λύση από AI — ελέγξτε την.

7

Συντακτική ανάλυση LL(1)

Μέθοδος

(μέθοδος με δικά μας λόγια)
Βήμα 1: αριστερή αναδρομή και κοινά προθέματα. Αν μια παραγωγή ξεκινά με το ίδιο μη τερματικό που ορίζει, υπάρχει άμεση αριστερή αναδρομή. Αν δύο εναλλακτικές ξεκινούν με το ίδιο σύμβολο, χρειάζεται αριστερή παραγοντοποίηση. Και στις δύο περιπτώσεις η γραμματική δεν είναι LL(1) όπως είναι.
Βήμα 2: σύνολα FIRST και FOLLOW. Το FIRST μιας εναλλακτικής είναι τα πρώτα τερματικά που μπορεί να παράγει. Το FOLLOW ενός μη τερματικού είναι τα σύμβολα που μπορεί να ακολουθούν αυτό στη συμβολοσειρά, συμπεριλαμβανομένου του $ για το τέλος.
Βήμα 3: έλεγχος. Η γραμματική είναι LL(1) αν για κάθε μη τερματικό τα FIRST των εναλλακτικών είναι ξένα μεταξύ τους, και αν μια εναλλακτική μπορεί να είναι το κενό, το FIRST των άλλων δεν τέμνει το FOLLOW.
Βήμα 4: ίχνος. Η στοίβα ξεκινά με $ και το αρχικό σύμβολο, και η είσοδος με τη συμβολοσειρά και $. Για κάθε βήμα: αν η κορυφή είναι μη τερματικό, κάνε πρόβλεψη με την εναλλακτική του πίνακα· αν είναι τερματικό, ταίριαξε με την είσοδο. Αν η είσοδος τελειώσει και η στοίβα αδειάσει, δέχεσαι.

Παράδειγμα: έλεγχος LL(1) και μετασχηματισμός αριστερής αναδρομής (Ιούνιος 2025, Παραλλαγή Α, θέμα 1, δ)

Θέμα εξέτασης

ΘΕΜΑ 1 – Συντακτική Ανάλυση (25%) Δίνεται η παρακάτω απλή BNF γραμματική, με τερματικά σύμβολα τα: ( ) [ ]

```
<S> ::= <S>(<S>) | <S>[<S>] | $\epsilon$
```

(d) Είναι ή όχι η γραμματική LL(1) και γιατί; Αν όχι, μετασχηματίστε την σε μια ισοδύναμη LL(1) γραμματική.

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Η λύση του αρχείου (ai-unverified) λέει ότι **δεν είναι LL(1)**, γιατί έχει άμεση αριστερή αναδρομή. Αυτό ισχύει: οι κανόνες `S → S(S)` και `S → S[S]` ξεκινούν με `S`.
Μετασχηματισμός (ελέγχθηκε εδώ με τα FIRST/FOLLOW):

$$S \to (S)S \mid [S]S \mid \epsilon$$

- FIRST((S)S) = (, FIRST([S]S) = [, FIRST(ε) = {ε}.
- FOLLOW(S) =),], $.
- Τα FIRST των εναλλακτικών είναι ξένα μεταξύ τους, και η εναλλακτική ε δεν συγκρούεται με το FOLLOW. Άρα η νέα γραμματική είναι LL(1).

Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να ξεχνάς ότι ο μετασχηματισμός πρέπει να δώσει **ισοδύναμη** γραμματική (τις ίδιες συμβολοσειρές). Μην αλλάξεις τη γλώσσα. Η αριστερή αναδρομή στα θέματα είναι συχνά η παγίδα: έλεγξε πρώτα την γραμματική και μετά κάνε τον πίνακα.

Παράδειγμα: αριστερή παραγοντοποίηση σε γραμματική με κοινό πρόθεμα (Ιούνιος 2026, θέμα 1, δ, μερική μεταγραφή)

Θέμα εξέτασης

Δίνεται η παρακάτω απλή BNF γραμματική, με τερματικά σύμβολα τα: 0 1

```
$$<S> ::= 0<A>0$$
$$<A> ::= 1<A>1 \mid 1011$$
```

(δ) Είναι ή όχι η γραμματική LL(1) και γιατί; Αν όχι, μετασχηματίστε την σε μια ισοδύναμη LL(1) γραμματική. Τι παρατηρείτε στη νέα LL(1) γραμματική;

Λύση

Χωρίς κλειδί. Η μέθοδος: η γραμματική αυτή **δεν** έχει αριστερή αναδρομή, οπότε ο έλεγχος είναι το FIRST των δύο εναλλακτικών του `<A>`. Η εναλλακτική `1<A>1` και η `1011` ξεκινούν με το ίδιο σύμβολο `1`. Άρα πρέπει να εφαρμοστεί αριστερή παραγοντοποίηση. Γράψε τη νέα γραμματική και έλεγξε ξανά FIRST και FOLLOW. Το ερώτημα ζητά και παρατήρηση για τη νέα γραμματική, οπότε γράψε τι αλλάζει στο FIRST.

Σημείωση

Συχνό λάθος: να απαντάς «ναι» ή «όχι» χωρίς τα σύνολα FIRST και FOLLOW. Οι διδάσκοντες βαθμολογούν τα βήματα. Στο ίχνος της στοίβας, σε κάθε γραμμή γράψε τη στοίβα, την είσοδο και την ενέργεια.

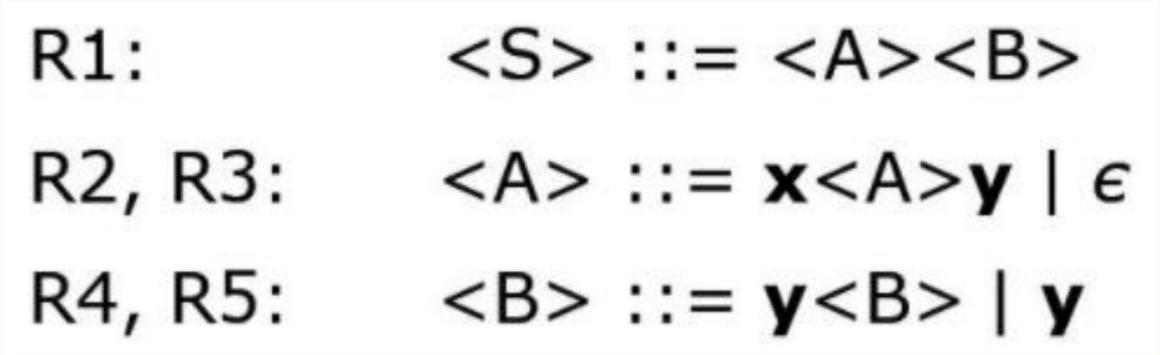
Ακόμη λυμένα θέματα

7.3.1 Σεπτέμβριος 2020, θέμα 1 (γραμματική R1–R5)

Θέμα εξέτασης

Δίνεται η BNF γραμματική που φαίνεται στο τέλος της εκφώνησης, με πέντε κανόνες R1 – R5 και τερματικά σύμβολα τα x y

a. Τι τύπου είναι η γραμματική στην ιεραρχία Chomsky; Είναι LL(1) η γραμματική; Αιτιολογήστε τις απαντήσεις σας.
b. Τι είδους συμβολοσειρές περιγράφει η παραπάνω γραμματική; Δώστε παραδείγματα αποδεκτών συμβολοσειρών, καθώς και τη μικρότερη σε μήκος.
c. Ελέγξτε αν η συμβολοσειρά xxyyyy είναι μέλος της γλώσσας, παρουσιάζοντας τη Στοίβα Ταιριάσματος – Πρόβλεψης Top-Down συντακτικής ανάλυσης της συμβολοσειράς.



Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή.

(a) Ο κανόνας R2 έχει το A ανάμεσα σε τερματικά, άρα η γραμματική δεν είναι κανονική: **Τύπου 2 (χωρίς συμφραζόμενα)**. **Δεν είναι LL(1)**: για το A ο έλεγχος περνά (FIRST(xAy) = {x}, και το ε χρειάζεται FOLLOW(A) = {y}, ξένα σύνολα), αλλά για το B οι `B → yB` και `B → y` ξεκινούν και οι δύο με `y`. Διορθώνεται με αριστερή παραγοντοποίηση: `B → yB'`, `B' → B | ε`.

(b) Το S παράγει $x^n y^n \cdot y^m$ με $n \geq 0$ και $m \geq 1$· άρα οι συμβολοσειρές έχουν πρώτα n `x`, και μετά $k = n + m > n$ `y`. Παραδείγματα: `y`, `yy`, `xyy`, `xxyyy`. Η μικρότερη είναι η `y`.

(c) Η `xxyyyy` ανήκει στη γλώσσα (n = 2, m = 2). Ίχνος (κορυφή στοίβας αριστερά):

Στοίβα	Είσοδος	Ενέργεια
S$	xxyyyy$	R1 (S → AB)
AB$	xxyyyy$	R2 (A → xAy)
xAyB$	xxyyyy$	ταίριασμα x
AyB$	xyyyy$	R2 (A → xAy)
xAyyB$	xyyyy$	ταίριασμα x
AyyB$	yyyy$	R3 (A → ε)
yyB$	yyyy$	ταίριασμα y, y
B$	yy$	R4 (B → yB), γιατί ακολουθεί και άλλο y
yB$	yy$	ταίριασμα y
B$	y$	R5 (B → y), γιατί είναι το τελευταίο y
y$	y$	ταίριασμα y
$	$	αποδοχή

Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να διαλέγεις τον κανόνα του B με ένα μόνο σύμβολο κοιτάγματος. Εδώ χρειάζεται να δεις αν ακολουθεί ακόμη `y`, γι' αυτό η γραμματική δεν είναι LL(1).

7.3.2 Ιούνιος 2021, θέμα 2 (γ)

Θέμα εξέτασης

Μία συνθηματική γλώσσα απλών προτάσεων, περιγράφεται από την παρακάτω *BNF* γραμματική, με τερματικά σύμβολα τα: **a b d S** όπου το **S** συμβολίζει τον χαρακτήρα «κενό» (space).

c. Είναι η γραμματική LL(1); Αιτιολογήστε την απάντησή σας, υποδεικνύοντας όλα τα σημεία της γραμματικής που δικαιολογούν τον ισχυρισμό σας. Αν η γραμματική δεν είναι LL(1), τι είδους μετασχηματισμοί και σε ποια σημεία πρέπει να γίνουν, ώστε να δημιουργηθεί μια ισοδύναμη LL(1) γραμματική; (δεν χρειάζεται να παρουσιάσετε τους ίδιους τους μετασχηματισμούς, ούτε την ισοδύναμη γραμματική)

$$< \text{πρόταση} > ::= < \text{λέξη} > | < \text{πρόταση} > \mathbf{S} < \text{λέξη} >$$

$$< \text{λέξη} > ::= < \text{συλλαβή} > | < \text{συλλαβή} >< \text{λέξη} >< \text{συλλαβή} >$$

$$< \text{συλλαβή} > ::= < \text{βάση} > | < \text{βάση} >< \text{σύμφωνο} > | \mathbf{a} < \text{βάση} > | \mathbf{a} < \text{σύμφωνο} >$$

$$< \text{βάση} > ::= < \text{σύμφωνο} > \mathbf{a}$$

$$< \text{σύμφωνο} > ::= \mathbf{b} \mid \mathbf{d}$$

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: μέτρια. Η ανάλυση αφορά τα σημεία που φαίνονται στη γραμματική· η ακριβής μορφή των μετασχηματισμών δεν ελέγχθηκε πλήρως. Η γραμματική **δεν είναι LL(1)**, για τρεις λόγους: 1. Η <πρόταση> ::= <πρόταση>S<λέξη> είναι **αριστερά αναδρομική**· ο προβλέπων αναλυτής θα έμπαινε σε άπειρη αναδρομή. 2. Η <λέξη> έχει εναλλακτικές με **κοινό πρόθεμα** <συλλαβή>, και η <συλλαβή> έχει εναλλακτικές με κοινό πρόθεμα <βάση> ή a. 3. Η <λέξη> έχει δύο κανόνες που ξεκινούν και οι δύο με <συλλαβή>, άρα έχουν κοινό FIRST, ενώ η <συλλαβή> με <βάση> και <βάση><σύμφωνο> έχει FIRST {b, d} και στις δύο εναλλακτικές.

Απαιτούνται δύο είδη μετασχηματισμών: απαλοιφή της αριστερής αναδρομής στην <πρόταση> και αριστερή παραγοντοποίηση στις <λέξη> και <συλλαβή>. Η εναλλακτική <λέξη> ::= <συλλαβή><λέξη><συλλαβή> μπορεί να μένει μη LL(1) και μετά τους μετασχηματισμούς, γιατί το FOLLOW περιέχει τα ίδια σύμβολα με το FIRST· αυτό δεν έχει ελεγχθεί πλήρως.

Λύση από AI — ελέγξτε την.

8

Λεξική ανάλυση: αυτόματα και κανονικές εκφράσεις

Μέθοδος

(μέθοδος με δικά μας λόγια)
Από την περιγραφή στο αυτόματο. Κάθε κατάσταση ενός ντετερμινιστικού αυτομάτου «θυμάται» κάτι για όσα έχει διαβάσει, π.χ. πόσα `1` έχουν εμφανιστεί ή αν το τελευταίο ψηφίο ήταν `1`. Ξεκίνα από την αρχική κατάσταση, γράψε μια κατάσταση για κάθε «πληροφορία» και πρόσθεσε μια κατάσταση-παγίδα (dead) για τις συμβολοσειρές που δεν μπορούν πια να γίνουν αποδεκτές.
Τελικές καταστάσεις. Τελική είναι κάθε κατάσταση στην οποία η συμβολοσειρά που διάβασες ανήκει στη γλώσσα. Αν το `ε` είναι αποδεκτό, η αρχική είναι τελική.
Από το αυτόματο στην έκφραση. Περιέγραψε τις διαδρομές από την αρχική στις τελικές καταστάσεις: ο βρόχος `(…)*` για κάθε επανάληψη, η εναλλακτική `|` για τα διαφορετικά μονοπάτια. Αν η περιγραφή έχει περιορισμούς («το πολύ τρία», «ακριβώς ένα»), οι περιορισμοί φαίνονται ως αντίστοιχος αριθμός καταστάσεων.
Έλεγχος. Δοκίμασε την έκφραση με 2–3 συμβολοσειρές που ανήκουν και μία που δεν ανήκει, και το αυτόματο με τις ίδιες.

Παράδειγμα: αυτόματο για περιορισμό στο πλήθος των άσσων (Ιούνιος 2025, Παραλλαγή Α, θέμα 2, α)

Θέμα εξέτασης

(a) Παρουσιάστε το διάγραμμα καταστάσεων – μεταβάσεων για το πεπερασμένο αυτόματο που αναγνωρίζει όλους τους δυαδικούς αριθμούς οι οποίοι περιέχουν το πολύ τρία 1. Το ϵ είναι αποδεκτός αριθμός.

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Η λύση του αρχείου (ai-unverified) ελέγχθηκε εδώ και είναι σωστή στην ουσία. Καταστάσεις q0, q1, q2, q3 μετρούν τους άσσους (0 έως 3)· οι q0–q3 είναι τελικές και η q4 είναι η κατάσταση-παγίδα για τέταρτο άσσο. Στις q0–q3 το `0` μένει στην ίδια κατάσταση και το `1` προχωρά στην επόμενη. Από την q4 όλα μένουν στην q4.
Για την κανονική έκφραση (β) της ίδιας άσκησης, το αρχείο δίνει $0^*(10^* + \epsilon)(10^* + \epsilon)(10^* + \epsilon)$, που ελέγχθηκε: κάθε γκρουπ «1 ακολουθούμενο από μηδενικά» μπορεί να εμφανιστεί το πολύ τρεις φορές.
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να ξεχνάς την κατάσταση-παγίδα. Χωρίς αυτήν, το αυτόματο δεν είναι πλήρες και ο έλεγχος «δεν ανήκει» γίνεται λάθος. Το `ε` ως αποδεκτή συμβολοσειρά σημαίνει ότι η αρχική κατάσταση είναι τελική.

Παράδειγμα: αυτόματο με περιορισμό στο 0 (Ιούνιος 2024, θέμα 2, α)

Θέμα εξέτασης

(α) Παρουσιάστε το *διάγραμμα καταστάσεων – μεταβάσεων* για το πεπερασμένο αυτόματο που αναγνωρίζει όλους τους δυαδικούς αριθμούς οι οποίοι: αν περιέχουν 1, κάθε ένα θα ακολουθείται από ένα ακριβώς 0, εκτός από το τελευταίο 1 που θα ακολουθείται από τουλάχιστον δύο 0. Ο αριθμός μπορεί να αρχίζει με ένα ή περισσότερα 0. Το ε είναι αποδεκτός αριθμός.

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Η λύση του αρχείου (ai-unverified) δίνει την κανονική έκφραση $0^*(10)^*100\ 0^*$. Το ελέγξαμε: δέχεται σωστά τα `100`, `10100`, `1000`, και απορρίπτει `10` και `1`. Όμως **δεν δέχεται** τις συμβολοσειρές χωρίς άσσο (π.χ. `0` ή το ε), που η εκφώνηση αποδέχεται ρητά. Για να καλυφθούν, πρέπει να προστεθεί η εναλλακτική 0^* (ή να γίνει το αυτόματο να δέχεται τις συμβολοσειρές χωρίς `1`).
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να ξεχνάς ότι μια συνθήκη του τύπου «αν περιέχει 1, τότε…» δεν περιορίζει τις συμβολοσειρές χωρίς `1`. Ο έλεγχος «το ε είναι αποδεκτό» είναι το καλύτερο τεστ για το αν ξέχασες την περίπτωση.

Παράδειγμα: αυτόματο με περιορισμό στην εμφάνιση του 1 (Ιούνιος 2026, θέμα 2, α, μερική μεταγραφή)

Θέμα εξέτασης

(α) Να παρουσιάσετε το *διάγραμμα καταστάσεων – μεταβάσεων* για το ντετερμινιστικό πεπερασμένο αυτόματο που αναγνωρίζει όλους τους δυαδικούς αριθμούς οι οποίοι: (i) περιέχουν ένα τουλάχιστον 1 και (ii) τα 0 (αν υπάρχουν) εμφανίζονται ένα τη φορά, έχοντας δεξιά και αριστερά τους ένα ή περισσότερα 1. Π.χ. οι αριθμοί `11` `110111101`.

Λύση

Χωρίς κλειδί. Η μέθοδος: το αυτόματο χρειάζεται καταστάσεις για «δεν έχω δει 1 ακόμη», «έχω δει 1 και το τελευταίο ήταν 1», «έχω δει μονό 0 ανάμεσα σε 1». Δοκίμασε μικρά παραδείγματα (`11`, `101`, `1001`) και σημείωσε ποια μονοπάτια τα δέχονται. Επειδή κάθε `0` πρέπει να έχει `1` και αριστερά και δεξιά, τα μηδενικά στην αρχή ή στο τέλος και τα δύο μηδενικά στη σειρά οδηγούν στην κατάσταση-παγίδα.

Σημείωση

Συχνό λάθος: να σχεδιάζεις το αυτόματο χωρίς να ελέγχεις τις συμβολοσειρές που **απορρίπτονται**. Για κάθε κατάσταση ρώτα «τι γίνεται με `0` και τι με `1`» και βεβαιώσου ότι υπάρχει μετάβαση και για τα δύο.

Ακόμη λυμένα θέματα

8.4.1 Ιούνιος 2021, θέμα 1 (a)–(c)

Θέμα εξέτασης

a. Να παρουσιάσετε τον *Πίνακα Καταστάσεων – Μεταβάσεων* (εναλλακτικός τρόπος παρουσίασης του διαγράμματος καταστάσεων – μεταβάσεων), για το ντετερμινιστικό πεπερασμένο αυτόματο που αναγνωρίζει όλους τους δυαδικούς αριθμούς οι οποίοι: Περιέχουν ένα τουλάχιστον **0**, ενώ τα **1** (αν υπάρχουν) εμφανίζονται ένα τη φορά και έχουν δεξιά και αριστερά τους δύο ή περισσότερα **0**. Παραδείγματα: **0**, **0000**, **0010000100001000**. Πόσες καταστάσεις έχει το αυτόματο; Ποιες από αυτές είναι τελικές καταστάσεις;
b. Δείξτε σε Πίνακα τη λειτουργία και το αποτέλεσμα του αυτόματου, για την είσοδο **001000**.
c. Γράψετε μια κανονική έκφραση που αντιστοιχεί στο παραπάνω αυτόματο.

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή.

(a) Καταστάσεις: q0 (τίποτα ακόμη), q1 (ακριβώς ένα 0 από την αρχή), q2 (δύο ή περισσότερα 0 στη σειρά), q3 (μόλις διαβάστηκε 1), q4 (1 και μετά ένα 0), και ν (νεκρή παγίδα). Τελικές: **q1 και q2**.

Κατάσταση	με 0	με 1	Τελική
q0	q1	ν	όχι
q1	q2	ν	**ναι**
q2	q2	q3	**ναι**
q3	q4	ν	όχι
q4	q2	ν	όχι
ν	ν	ν	όχι

Το αυτόματο έχει έξι καταστάσεις, αν μετράμε και τη νεκρή.

(b) Για την είσοδο `001000`: q0 → q1 → q2 → q3 → q4 → q2 → q2. Τέλος εισόδου στην q2, που είναι τελική· άρα **αποδοχή**.

(c) Με Z = 00 0* (δύο ή περισσότερα μηδενικά):

$$0 \mid 00\,0^{*}\,(\,1\,00\,0^{*}\,)^{*}$$

Το `0010000100001000` ταιριάζει: `00` · `1` · `000` · `1` · `0000` · `1` · `000`.

Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να ξεχνάς την κατάσταση-παγίδα. Χωρίς αυτήν, το αυτόματο δεν είναι πλήρες και ο έλεγχος «δεν ανήκει» γίνεται λάθος.

8.4.2 Ιούνιος 2022, θέμα 1 (e)

Θέμα εξέτασης

Δίνεται η παρακάτω BNF γραμματική με αλφάβητο a b

$$< S >::= a < A > | \; b < B >$$

$$< A >::= a < S > | \; a$$

$$< B >::= b < S > | \; b$$

(d) Μπορείτε να φτιάξετε μια *κανονική έκφραση* αντιστοιχη της γραμματικής και γιατί; Αν ναι, παρουσιάστε την.

(e) Επιπλέον, υπάρχει ντετερμινιστικό πεπερασμένο αυτόματο αντιστοιχο της κανονικής έκφρασης; Αν ναι, παρουσιάστε το *διάγραμμα καταστάσεων – μεταβάσεων* του αυτόματου.

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή. Το αυτόματο έχει πέντε καταστάσεις (q0 η αρχική, qa, qb, qf η τελική, ν η νεκρή).

Κατάσταση	με a	με b
q0 (αρχή, μη τελική)	qa	qb
qa (είδαμε a)	qf	ν
qb (είδαμε b)	ν	qf
qf (ολοκληρώθηκε ζεύγος, τελική)	qa	qb
ν	ν	ν

Η qf συμπεριφέρεται όπως η q0 αλλά είναι τελική· γι' αυτό δεν συγχωνεύονται. Το διάγραμμα είναι ο ίδιος πίνακας σε μορφή κόμβων και βελών.
Λύση από AI — ελέγξτε την.

9

Συναρτησιακός: στρατηγικές αποτίμησης

Μέθοδος

(μέθοδος με δικά μας λόγια· βλ. διαφάνειες για την παρουσίαση)
Αναγωγή. Μια έκφραση αποτιμάται επαναλαμβάνοντας την αντικατάσταση ενός **redex** (υποέκφρασης που μπορεί να αναχθεί) με το ισοδύναμο της από κανόνες ή από ενσωματωμένες πράξεις. Η έκφραση χωρίς redex είναι η **κανονική μορφή**. Το ίδιο αποτέλεσμα μπορεί να βγει από διαφορετικές διαδρομές· ο αριθμός των βημάτων διαφέρει.
Πρόθυμη (eager). Πρώτα αποτιμάς τα ορίσματα και μετά εφαρμόζεις τη συνάρτηση. Ένα όρισμα που εμφανίζεται δύο φορές υπολογίζεται μία φορά.
Οκνηρή (lazy). Εφαρμόζεις πρώτα τον εξωτερικό κανόνα χωρίς να αποτιμήσεις τα ορίσματα. Αν ένα όρισμα εμφανίζεται πολλές φορές και δεν μοιράζεσαι το αποτέλεσμα, θα αποτιμηθεί πολλές φορές. Η **γραφο-υβριδική** (outermost graph reduction) μοιράζεται τα κοινά υποδέντρα, γι' αυτό είναι συχνά η πιο αποδοτική.
Έλεγχος. Για κάθε στρατηγική γράψε πρώτα τη σειρά των redex και μετά μέτρα τα βήματα. Αν μια συνάρτηση έχει κανόνα `sum (x:xs) = …` με αναδρομή, μέτρα και τα βήματα των επαναλήψεων.

Παράδειγμα: πρόθυμη και οκνηρή αποτίμηση του `square(sum ...)` (Σεπτέμβριος 2023, θέμα 6, μερική μεταγραφή)

Θέμα εξέτασης

Σε μία γλώσσα συναρτησιακού προγραμματισμού, θεωρείστε τους κάτωθι κανόνες αναγωγής (reduction rules).

- Built-in rules: addition, substraction, multiplication, division
- User supplied rules:

```
square x = x * x
double x = x + x
sum [ ] = 0
sum (x:xs) = x + sum [x+1:xs]
```

Δώστε όλα τα ενδιάμεσα βήματα αναγωγής μέχρι την τελική αποτίμηση της έκφρασης `square (sum [1:10])` με πρόθυμη (eager evaluation) και οκνηρή (lazy evaluation) στρατηγική αποτίμησης. Πόσα βήματα (reductions) απαιτεί η κάθε στρατηγική; Ποια είναι η πιο αποδοτική;

Λύση

Χωρίς κλειδί. Το αρχείο λύσεων (ai-unverified) χρησιμοποιεί τον κανόνα `sum (x:xs) = x + sum xs`, ενώ η εκφώνηση γράφει `sum [x+1:xs]`. Γι' αυτό η λύση του αρχείου δεν χρησιμοποιείται. Η μέθοδος: στην πρόθυμη στρατηγική το `sum [1:10]` αποτιμάται πλήρως πριν από το `square`, άρα μία φορά. Στην οκνηρή, το `square` ανοίγει πρώτα, και το `sum [1:10]` εμφανίζεται δύο φορές. Αν δεν υπάρχει μοίρασμα, κάθε εμφάνιση παράγει ξεχωριστά τα βήματά της. Γράψε κάθε βήμα με τον κανόνα που εφαρμόζεις.

Σημείωση

Συχνό λάθος: να ξεχνάς ότι ο κανόνας της εκφώνησης μπορεί να αλλάζει τη λίστα σε κάθε αναδρομή (`[x+1:xs]`). Αντιγράφοντας λύση που χρησιμοποιεί άλλο κανόνα παίρνεις λάθος αριθμό βημάτων. Το ερώτημα «ποια είναι η πιο αποδοτική» απαντιέται με τα βήματα, όχι με τη διαίσθηση.

Παράδειγμα: λ-λογισμός με τρία ορίσματα (Ιούνιος 2025, Παραλλαγή Α, θέμα 6, ε)

Θέμα εξέτασης

(ε) Συμπληρώστε όλα τα στάδια αποτίμησης της παρακάτω έκφρασης σε λ-λογισμό:

$$(\lambda x.\, \lambda y.\, \lambda z.\, x^3 + y^3 + z^3)\ 3\ 2\ 1$$

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Το αρχείο λύσεων (ai-unverified) δίνει τα στάδια: αντικατάσταση του x με 3 → $(\lambda y.\lambda z.\, 27 + y^3 + z^3)\ 2\ 1$· αντικατάσταση του y με 2 → $(\lambda z.\, 35 + z^3)\ 1$· αντικατάσταση του z με 1 → $35 + 1 = 36$. Ελέγχθηκε: $3^3 + 2^3 + 1^3 = 27 + 8 + 1 = 36$.
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να αντικαθιστάς δύο μεταβλητές μαζί. Στο λ-λογισμό κάθε εφαρμογή δεσμεύει μία μεταβλητή, με τη σειρά των ορισμάτων. Γράψε κάθε στάδιο ξεχωριστά.

Παράδειγμα: λ-λογισμός με λογικές τιμές (Σεπτέμβριος 2022, θέμα 5, α, μερική μεταγραφή)

Θέμα εξέτασης

(α) Στον λ-λογισμό, μπορούμε να αναπαραστήσουμε τις λογικές τιμές true και false ως εξής:

$$\text{true} = \lambda x.\, \lambda y.\, x$$

$$\text{false} = \lambda x.\, \lambda y.\, y$$

Η αναπαράσταση του λογικού τελεστή not με την έκφραση $\text{not} = \lambda z.\, z\ \text{false}\ \text{true}$, επιφέρει το επιθυμητό αποτέλεσμα;

Λύση

Χωρίς κλειδί. Η μέθοδος: για το `not true` αντικατάστησε το `z` με το `true` και αποτίμησε `true false true`, και για το `not false` το ίδιο με το `false`. Κάθε αποτίμηση είναι μία ή δύο αντικαταστάσεις. Αν το αποτέλεσμα είναι `false` και `true` αντίστοιχα, η αναπαράσταση δουλεύει. Πρόσεξε τη σειρά των μεταβλητών στο `true` και στο `false`.

Σημείωση

Συχνό λάθος: να μπερδεύεις το `false` με το `0` της C. Εδώ το `false` είναι συνάρτηση που επιλέγει το **δεύτερο** όρισμα. Όταν αποτιμάς, γράψε ρητά ποιο όρισμα επιλέγεται.

Ακόμη λυμένα θέματα

9.4.1 Σεπτέμβριος 2024, θέμα 4 (δ)–(ε)

Θέμα εξέτασης

(δ) Συμπληρώστε όλα τα στάδια αποτίμησης της παρακάτω έκφρασης σε λ-λογισμό.

$$(\lambda x.\, \lambda y.\, \lambda z.\, x + y + z)\ 4\ 5\ 6$$

(ε) Σε μία γλώσσα συναρτησιακού προγραμματισμού, θεωρείστε τους κάτωθι κανόνες αναγωγής (reduction rules). - Built-in rules: addition, substraction, multiplication, division - User supplied rules: - square x = x * x - double x = x + x - sum [ ] = 0 - sum (x:xs) = x + sum [x+1:xs]
Δώστε όλα τα ενδιάμεσα βήματα αναγωγής μέχρι την τελική αποτίμηση της έκφρασης square (sum [1:10]) με πρόθυμη (eager evaluation) και οκνηρή (lazy evaluation) στρατηγική αποτίμησης. Πόσα βήματα (reductions) απαιτεί η κάθε στρατηγική; Ποια είναι η πιο αποδοτική;

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή για το (δ), μέτρια για το (ε). Δεν επιβεβαιώθηκε (ε): το πλήθος βημάτων εξαρτάται από τη σύμβαση μέτρησης. Στις διαφάνειες του μαθήματος (κεφ. 16) για το `square (sum [1..100])` δίνονται 102 αναγωγές (πρόθυμη) και 202 (οκνηρή χωρίς διαμοιρασμό), δηλαδή μετράται μία αναγωγή ανά εφαρμογή του `sum`· με την ίδια σύμβαση το `square (sum [1:10])` θα έδινε 12 και 22. Οι αριθμοί 33 και 64 παρακάτω ακολουθούν πιο αναλυτική μέτρηση· το συμπέρασμα για το ποια στρατηγική είναι πιο αποδοτική είναι το ίδιο.
(δ) $(\lambda x.\lambda y.\lambda z.\, x + y + z)\ 4\ 5\ 6 \to (\lambda y.\lambda z.\, 4 + y + z)\ 5\ 6 \to (\lambda z.\, 4 + 5 + z)\ 6 \to 4 + 5 + 6 \to 15$.
(ε) Η εκφώνηση ορίζει `sum (x:xs) = x + sum [x+1:xs]`. Ερμηνεύουμε το `[m:n]` ως τη λίστα ακεραίων από m έως n, με `sum [] = 0`. Η ερμηνεία αυτή είναι αβέβαιη, και ο αριθμός των βημάτων εξαρτάται από τη σύμβαση μέτρησης. Με τη σύμβαση «κάθε εφαρμογή κανόνα ή πράξης είναι μία αναγωγή», για το `sum [1:10]` μετρώνται 10 εφαρμογές του `sum`, 10 υπολογισμοί του `x+1`, 1 τερματισμός και 10 προσθέσεις: 31 αναγωγές, αποτέλεσμα 55.

- *Πρόθυμη:* πρώτα το όρισμα (31 αναγωγές), μετά `square 55` → 55·55 = 3025 (2 αναγωγές). **Σύνολο 33.**
- *Οκνηρή χωρίς διαμοιρασμό:* `square (sum [1:10])` → `sum [1:10] * sum [1:10]`, και το `sum` υπολογίζεται δύο φορές (2·31). **Σύνολο 64** (1 + 62 + 1).
- *Οκνηρή με διαμοιρασμό (graph reduction):* το `sum` υπολογίζεται μία φορά: 1 + 31 + 1 = **33**.

Η πιο αποδοτική είναι η πρόθυμη ή η οκνηρή με διαμοιρασμό (33 αναγωγές). Η οκνηρή χωρίς διαμοιρασμό κάνει διπλή δουλειά.
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να ξεχνάς ότι στο λ-λογισμό κάθε **λ** δεσμεύει μία μεταβλητή, με τη σειρά των ορισμάτων. Γράψε κάθε βήμα χωριστά.

10

Map, reduce και λογικός προγραμματισμός

Μέθοδος

(μέθοδος με δικά μας λόγια)
Map. Η `map(f, L)` εφαρμόζει τη συνάρτηση σε κάθε στοιχείο και επιστρέφει νέα ακολουθία. Στην Python επιστρέφει αντικείμενο `map`, οπότε χρειάζεται `list(...)` ή `sum(...)` για να φανεί το αποτέλεσμα.
Reduce. Η `functools.reduce(f, L)` συνδυάζει τα στοιχεία από αριστερά προς τα δεξιά με δυαδική συνάρτηση, ενώ το τρίτο όρισμα (αρχική τιμή) ξεκινά τον συνδυασμό. Για άθροισμα τετραγώνων συχνά γράφεις `reduce(lambda a, b: a + b*b, L, 0)`.
Μία βασική εντολή. Όταν η εκφώνηση ζητά «μία και μόνο βασική εντολή», γράψε μία έκφραση που περιέχει τη `map` ή τη `reduce` και μέσα της το `lambda`. Το `print(...)` μετράει ως εντολή εξόδου, οπότε μπορεί να συμπεριληφθεί.
Prolog. Τα facts είναι σχέσεις, οι κανόνες είναι υποθέσεις που συνεπάγονται κάτι (το `:-` διαβάζεται «αν»), και τα ερωτήματα ψάχνουν τιμές για τις μεταβλητές (με κεφαλαίο). Ο κανόνας `professor(X, Y) :- teaches(X, C), studies(Y, C).` διαβάζεται: ο X είναι καθηγητής του Y αν ο X διδάσκει ένα μάθημα C και ο Y σπουδάζει το ίδιο C.

Παράδειγμα: άθροισμα κύβων με lambda και map (Ιούνιος 2024, θέμα 6, α)

Θέμα εξέτασης

(α) Κάνοντας χρήση της Python lambda map function, γράψτε πρόγραμμα το οποίο με μία και μόνο βασική εντολή, θα υπολογίζει (και τυπώνει) το άθροισμα των κύβων των ακεραίων αριθμών μιας λίστας L.
(β) Κάνοντας χρήση της Python lambda reduce function, γράψτε πρόγραμμα το οποίο με μία και μόνο βασική εντολή, θα υπολογίζει (και τυπώνει) το άθροισμα των τετραγώνων των ακεραίων αριθμών μιας λίστας L.

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Η λύση του αρχείου (ai-unverified) για το (α) είναι η εντολή `print(sum(map(lambda x: x**3, L)))`, που είναι σωστή. Για το (β) το αρχείο δίνει `reduce(lambda a, b: a + b**2, [0] + L)` (με αρχική τιμή μέσα στη λίστα). Η έκφραση είναι σωστή και εδώ, γιατί το `0` είναι ουδέτερο στην πρόσθεση, αλλά για καθαρότητα προτιμάμε την αρχική τιμή ως τρίτο όρισμα.
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να γράφεις `map` με κενή τιμή ή να ξεχνάς ότι η `reduce` σε άδεια λίστα χρειάζεται αρχική τιμή. Το `map` στην Python 3 δεν τυπώνει μόνο του· χρειάζεται `print` ή `list`.

Παράδειγμα: τι τυπώνει ένα πρόγραμμα με reduce (Ιούνιος 2025, Παραλλαγή Α, θέμα 6, γ)

Θέμα εξέτασης

(γ) Τι τυπώνει το παρακάτω πρόγραμμα;

```
import functools
numbers = [1, 2, 3, 4]
result = functools.reduce(lambda x, y: x * y, numbers)
print(result)
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Το αρχείο λύσεων (ai-unverified) δίνει την αλυσίδα: `1·2 = 2, 2·3 = 6, 6·4 = 24`, άρα τυπώνει **24**. Ελέγχθηκε.
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: για το γινόμενο των k-στών δυνάμεων (δ) δεν αρκεί να αλλάξεις τη συνάρτηση της `reduce`. Πρέπει να υψώσεις κάθε στοιχείο στη k-στή δύναμη και μετά να εφαρμόσεις πολλαπλασιασμό.

Σημείωση: ερώτημα Prolog (Ιούνιος 2026, θέμα 6, Α, μερική μεταγραφή)

Θέμα εξέτασης

(Α) Δίνονται σε γλώσσα *Prolog* οι παρακάτω 8 προτάσεις (facts) καθώς και ένας κανόνας (rule):
Facts: `studies(Charlie, csc135)`, `studies(Olivia, csc135)`, `studies(Jack, csc131)`, `studies(Arthur, csc134)`, `teaches(Sioutas, csc135)`, `teaches(Garofalakis, csc131)`, `teaches(Hadjidoukas, csc171)`, `teaches(Vonitsanos, csc134)`
Rules: `professor(X, Y) :- teaches(X, C), studies(Y, C).`
(γ) Γράψτε σε γλώσσα Prolog τα ακόλουθα ερωτήματα: (γ1) Τι μελετάει ο Charlie `//charlie studies what?//` (γ2) Ποιοι είναι οι μαθητές του Καθηγητή Garofalakis `// Who are the students of professor Garofalakis?//`

Λύση

Χωρίς κλειδί. Η μέθοδος: σχεδίασε τα facts ως δέντρα (`studies(Charlie, csc135)` είναι ρίζα με δύο παιδιά) και διάβασέ τα ως προτάσεις («ο Charlie σπουδάζει το csc135»). Για τον κανόνα, διάβασε το `:-` ως «αν», και τα ερωτήματα ως «για ποιες τιμές του X ισχύει». Το (γ2) ζητά όλους τους μαθητές του Garofalakis, άρα η σχέση με το `csc131` και τη σχέση `studies` με το ίδιο μάθημα.

Σημείωση

Συχνό λάθος: στην Prolog τα ονόματα με κεφαλαίο αρχικό είναι **μεταβλητές**. Η εκφώνηση γράφει `Charlie`, `Olivia` και άλλα με κεφαλαίο, οπότε στην απάντησή σου γράψε τις σταθερές με μικρό

αρχικό (`charlie`) και σημείωσε την ασάφεια αν σε ρωτήσουν.

Ακόμη λυμένα θέματα

10.4.1 Σεπτέμβριος 2024, θέμα 4 (γ)

Θέμα εξέτασης

(γ) Έστω σε Lisp οι παρακάτω Map/ Fold (Reduce) high-order functions. Συμπληρώστε τις αποτιμήσεις ? των εκφράσεων.

```
(map (lambda (x) (* x 3))
    '(1 2 3 4 5)) => ?
(fold + 0 '(1 2 3 4 5)) => ?
(fold * 1 '(1 2 3 4 5)) => ?
(define (sum-of-triples v)
(fold + 0 (map (lambda (x) (* x 3)) v)))
(sum-of-triples '(1 2 3 4 5)) => ?
```

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή.

- `map` με `(* x 3)` στο `(1 2 3 4 5)` → **(3 6 9 12 15)**.
- `(fold + 0 '(1 2 3 4 5))` → **15**.
- `(fold * 1 '(1 2 3 4 5))` → **120**.
- `(sum-of-triples '(1 2 3 4 5))` → το `map` δίνει (3 6 9 12 15) και το `fold +` δίνει **45**.

Λύση από AI — ελέγξτε την.

Σημείωση

Η `fold` παίρνει αρχική τιμή ως δεύτερο όρισμα. Για άθροισμα ή γινόμενο η αρχική τιμή είναι το ουδέτερο στοιχείο της πράξης (0 για την πρόσθεση, 1 για τον πολλαπλασιασμό).

10.4.2 Ιούνιος 2026, θέμα 6 (Α), ερωτήματα (α)–(β)

Θέμα εξέτασης

(Α) Δίνονται σε γλώσσα *Prolog* οι παρακάτω 8 προτάσεις (facts) καθώς και ένας κανόνας (rule):
Facts: `studies(Charlie, csc135)`, `studies(Olivia, csc135)`, `studies(Jack, csc131)`, `studies(Arthur, csc134)`, `teaches(Sioutas, csc135)`, `teaches(Garofalakis, csc131)`, `teaches(Hadjidoukas, csc171)`, `teaches(Vonitsanos, csc134)`
Rules: `professor(X, Y) :- teaches(X, C), studies(Y, C).`
(α) Σχεδιάστε τις δομές δεδομένων (δέντρα) για τις παραπάνω 8 προτάσεις. Πώς «διαβάζονται» οι παραπάνω προτάσεις;

(β) Οι κανόνες είναι θεωρήματα που επιτρέπουν στο διερμηνέα να συναγάγει την αλήθεια προτάσεων. Πως διαβάζεται ο παραπάνω κανόνας;

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή για το (β), μέτρια για τη μορφή των δέντρων.

(α) Κάθε πρόταση είναι δέντρο με ρίζα το κατηγόρημα και παιδιά τα ορίσματα. Στην Prolog τα ονόματα με κεφαλαίο είναι μεταβλητές· γράφουμε τις σταθερές με μικρό αρχικό:

```
   studies          studies          studies          studies
   /     \          /     \          /     \          /     \
charlie csc135   olivia csc135    jack  csc131    arthur csc134

   teaches          teaches          teaches          teaches
   /     \          /     \          /     \          /     \
sioutas csc135  garofalakis csc131 hadjidoukas csc171 vonitsanos csc134
```

Ανάγνωση: «ο charlie σπουδάζει το csc135», «η olivia σπουδάζει το csc135», «ο jack το csc131», «ο arthur το csc134», «ο sioutas διδάσκει το csc135», «ο garofalakis διδάσκει το csc131», «ο hadjidoukas διδάσκει το csc171», «ο vonitsanos διδάσκει το csc134».

(β) Ο κανόνας διαβάζεται: «ο X είναι καθηγητής του Y **αν** υπάρχει μάθημα C τέτοιο ώστε ο X διδάσκει το C **και** ο Y σπουδάζει το C». Το `:-` σημαίνει «αν» και το κόμμα σημαίνει «και».

Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να γράφεις σταθερές με κεφαλαίο αρχικό. Στην Prolog το `Charlie` είναι μεταβλητή και το `charlie` είναι σταθερά.

10.4.3 Ιούνιος 2026, θέμα 6 (Γ), ερώτημα (β)

Θέμα εξέτασης

(Γ) (α) Κάνοντας χρήση της *Python lambda map function*, γράψτε πρόγραμμα το οποίο **με μία και μόνο βασική εντολή**, θα υπολογίζει (και τυπώνει) το άθροισμα των κύβων των ακέραιων αριθμών μιας λίστας L.

(β) Κάνοντας χρήση της *Python lambda reduce function*, γράψτε πρόγραμμα το οποίο **με μία και μόνο βασική εντολή**, θα υπολογίζει (και τυπώνει) το άθροισμα των τετραγώνων των ακέραιων αριθμών μιας λίστας L.

Λύση

AI **Δεν έχει ελεγχθεί από άνθρωπο — ελέγξτε την.** Εμπιστοσύνη: υψηλή.

```python
from functools import reduce
print(reduce(lambda a, b: a + b**2, L, 0))
```

Η εντολή `print(...)` είναι η μία βασική εντολή· το `import` μένει έξω από αυτήν. Η αρχική τιμή `0`

είναι απαραίτητη: χωρίς αυτήν το πρώτο στοιχείο της λίστας δεν υψώνεται στο τετράγωνο. Για L = [2, 3] η σωστή έξοδος είναι 13 (0 + 4 + 9).
Λύση από AI — ελέγξτε την.

Σημείωση

Συχνό λάθος: να γράφεις `reduce` χωρίς αρχική τιμή και να μην προσέχεις τι γίνεται με το πρώτο στοιχείο. Η `map` στην Python 3 επιστρέφει αντικείμενο `map`, οπότε χρειάζεται `list(...)` ή `sum(...)` για να φανεί το αποτέλεσμα.

11

Ασκήσεις εξάσκησης

Λύσε τα παρακάτω σαν να είσαι στην εξέταση, χωρίς σημειώσεις.

Σημάνσεις: ✓ λύση επίσημη ή κοινοτική (αναφέρεται το αρχείο λύσεων)· ⚙ λύση γραμμένη με τη βοήθεια AI, που δεν έχει ελεγχθεί από άνθρωπο (αρχείο λύσεων ai-unverified ή λύση μέσα στο κεφάλαιο)· ○ χωρίς λύση (συμπεριλαμβάνονται τα «Δεν λύθηκε» και τα θέματα χωρίς κλειδί). Όπου η εμπιστοσύνη είναι χαμηλή, το σημειώνω.

Δείκτες C και διαχείριση μνήμης (κεφ. 1)

- Σεπτέμβριος 2024, θέμα 3 — δείκτες σε δύο πίνακες, αριθμητική δεικτών · ⚙ (στο κεφάλαιο)
- Ιούνιος 2026, θέμα 3 — πίνακας με δείκτη και διαίρεση · ⚙ (στο κεφάλαιο)
- Ιούνιος 2025, Παραλλαγή Α, θέμα 4 — δείκτες, τελεστές αύξησης και σύνθετη ανάθεση · ⚙ (`2025-06-a.solutions`
- Ιούνιος 2025, Παραλλαγή Β, θέμα 4 — `p = &p[1]` και `(*(p-2))++` · ○ (ίδιο μοτίβο με την Α)
- Ιούνιος 2024, θέμα 4 — δείκτες με `(*(p-2))--` · ⚙ (`2024-06.solutions.md`, με προειδοποίηση για το πρόσημο)
- Σεπτέμβριος 2023, θέμα 4 (α) — τιμές μετά από τριαδικό τελεστή με `++flag` · ⚙ (`2023-09.solutions.md`)
- Ιούνιος 2023, Παραλλαγή Α, θέμα 2 — ίχνος πίνακα `data` με δείκτες · ⚙ (`2023-06-a.solutions.md`, μόνο οι τελικές τιμές)
- Ιούνιος 2023, Παραλλαγή Β, θέμα 2 — τελεστές αύξησης και δείκτες · ⚙ (`2023-06-b.solutions.md`, στο κεφάλαιο)
- Σεπτέμβριος 2022, θέμα 2 — στατικός πίνακας, μέγεθος μνήμης, μη αρχικοποιημένος δείκτης · ⚙ (υψηλή, το (α) μέτρια)
- Ιούνιος 2022, θέμα 3 (α) — τελεστές `++`, `--` και short-circuit στο `&&` · ⚙
- Ιούνιος 2022, θέμα 3 (β) — δείκτες σε πίνακα, `q = p + 2` (C++) · ⚙ (στο κεφάλαιο)

Μεταβίβαση παραμέτρων και εμβέλεια (κεφ. 2)

- Ιούνιος 2026, θέμα 4 — κλήση `Q(b[m], b[k])` με τέσσερις τρόπους · ⚙ (στο κεφάλαιο)
- Σεπτέμβριος 2024, θέμα 2 — διαδικασίες `F` και `G` με `a[i]`, τέσσερις τρόποι · ⚙ (στο κεφάλαιο)
- Ιούνιος 2024, θέμα 3 — διαδικασία `F(x, y, z)` με καθολικές μεταβλητές · ○
- Ιούνιος 2023, Παραλλαγή Α, θέμα 3 — κλήση με τιμή και αναφορά σε Pascal · ⚙ (`2023-06-a.solutions.md`)
- Ιούνιος 2023, Παραλλαγή Β, θέμα 3 — κλήση με τιμή σε Pascal με καθολικές · ⚙ (στο κεφάλαιο, μόνο κλήση με τιμή)
- Ιούνιος 2025, Παραλλαγή Α, θέμα 3 — εμβέλεια και τέσσερις τρόποι · ○ (χωρίς κλειδί)
- Ιούνιος 2025, Παραλλαγή Β, θέμα 3 — ίδια ιδέα, με τυπογραφικό λάθος στην τρίτη περίπτωση · ○
- Ιούνιος 2021, θέμα 3 — δύο προγράμματα με δείκτη και `a[n+3]` · ⚙ (μέτρια, στο κεφάλαιο για το (δ))
- Σεπτέμβριος 2021, θέμα 3 — παρόμοια δύο προγράμματα · ⚙ (μέτρια)
- Ιούνιος 2022, θέμα 2 — συνάρτηση `f(x, y, Lo, Hi)` με πίνακα `A` · ⚙ (χαμηλή)
- Σεπτέμβριος 2022, θέμα 4 — ένθετες `P` και `Q` με αναδρομική κλήση · ⚙ (χαμηλή)
- Σεπτέμβριος 2020, θέμα 2 (Ι) — ένθετες διαδικασίες, στατική εμβέλεια · ⚙
- Σεπτέμβριος 2020, θέμα 2 (ΙΙ) — δυναμική εμβέλεια στο ίδιο πρόγραμμα · ⚙ (μέτρια)
- Σεπτέμβριος 2020, ερώτηση 4 (πολλαπλής επιλογής) — τοπική και καθολική μεταβλητή · ⚙
- Σεπτέμβριος 2020, ερώτηση 5 (πολλαπλής επιλογής) — σειρά αποτίμησης ορισμάτων στην `print` · ⚙

Python (κεφ. 3)

- Σεπτέμβριος 2022, θέμα 3 (α) — `range(12, 2, -2)` με `break` · ⚙ (στο κεφάλαιο)
- Σεπτέμβριος 2022, θέμα 3 (β) — `x*x` σε συμβολοσειρά (`TypeError`) · ⚙
- Σεπτέμβριος 2022, θέμα 3 (γ) — `while` με `continue` και `break` · ⚙ (στο κεφάλαιο)
- Σεπτέμβριος 2020, ερώτηση 2 (πολλαπλής επιλογής) — λεξικό με `sorted` και `for … else` · ⚙
- Σεπτέμβριος 2020, ερώτηση 3 (πολλαπλής επιλογής) — διαίρεση με μηδέν · ⚙
- Ιούνιος 2022, θέμα 4 — συνάρτηση με `print(a, end='')` και λίστα · ⚙ (χαμηλή, μερικό κείμενο)
- Ιούνιος 2023, Παραλλαγή Β, θέμα 5 — ηλεκτρονικό καλάθι με συναρτήσεις · ○
- Ιούνιος 2024, θέμα 5 — δέκα ακέραιοι σε λίστα: μέγιστο, ελάχιστο, μέσος, αντιστροφή, διπλασιασμός, ταξινόμηση · ⚙ (`2024-06.solutions.md`, χωρίς κώδικα)
- Σεπτέμβριος 2023, θέμα 5 — παραγγελιοληψία σε εστιατόριο με μενού και απόδειξη · ⚙ (`2023-09.solutions.md`, με προειδοποίηση για τον κώδικα)
- Σεπτέμβριος 2024, θέμα 5 — λίστα αριθμών: εισαγωγή, διαγραφή, ταξινόμηση, επιλογή, αντικατάσταση · ⚙ (στο κεφάλαιο)
- Ιούνιος 2025, Παραλλαγή Α, θέμα 5 — λίστα φοιτητών με βαθμούς σε δύο μαθήματα (pandas) · ○
- Ιούνιος 2025, Παραλλαγή Β, θέμα 5 — το ίδιο θέμα · ○ (στο κεφάλαιο, χωρίς κλειδί)
- Ιούνιος 2026, θέμα 5 — `concat`, `merge`, `drop`, `iterrows` · ⚙ (μέτρια, στο κεφάλαιο)

Γραμματικές BNF και ιεραρχία Chomsky (κεφ. 4)

- Ιούνιος 2015, θέμα 3 — δεξιά γραμμική γραμματική, γλώσσα, bottom-up · ⚙
- Φεβρουάριος 2021, θέμα 1 — γραμματική R1–R7: τύπος, γλώσσα, κανονική έκφραση, ίχνος · ⚙ (Α–Γ στο κεφάλαιο)
- Ιούνιος 2021, θέμα 2 (α)–(γ) — συνθηματική γλώσσα με a, b, d, S · ⚙ (μέτρια· το (γ) στο κεφ. 5)
- Ιούνιος 2022, θέμα 1 (a)–(d) — `S ::= aA | bB`, bottom-up, τύπος, κανονική έκφραση · ⚙ (γ–δ στο κεφάλαιο)
- Ιούνιος 2023, Παραλλαγή Α, θέμα 1 (α) — περιγραφή κατασκευών με δηλώσεις · ⚙ (`2023-06-a.solutions.md`)
- Ιούνιος 2023, Παραλλαγή Β, θέμα 1 (δ) — τύπος Chomsky · ⚙ (στο κεφάλαιο)
- Ιούνιος 2024, θέμα 1 — γραμματική `<S> ::= <A><B><C><S> | d`, bottom-up για το `acd` · ⚙ (`2024-06.solutions.md`)
- Σεπτέμβριος 2024, θέμα 1 (a)–(d) — τρεις γραμματικές, αντιστοίχιση, κανονικότητα · ⚙ (το (β) στο κεφάλαιο)
- Ιούνιος 2026, θέμα 1 (α)–(β) — γραμματική `0<A>0`, τύπος και γλώσσα · ⚙ (στο κεφάλαιο)
- Σεπτέμβριος 2020, θέμα 1 (R1–R7) — γραμματική R1–R7: τύπος, γλώσσα, κανονική έκφραση · ⚙

LL(1) (κεφ. 5)

- Σεπτέμβριος 2020, θέμα 1 (R1–R5) — έλεγχος LL(1) και ίχνος για `xxyyyy` · ⚙ (στο κεφάλαιο)
- Ιούνιος 2021, θέμα 2 (γ) — έλεγχος LL(1) της συνθηματικής γλώσσας · ⚙ (μέτρια, στο κεφάλαιο)
- Ιούνιος 2022, θέμα 1 (f) — LL(1) και παραγοντοποίηση · ⚙
- Σεπτέμβριος 2024, θέμα 1 (f) — LL(1) και μετασχηματισμός της πρώτης γραμματικής · ⚙
- Φεβρουάριος 2021, θέμα 1 (Δ) — ίχνος top-down της `011101` · ⚙
- Ιούνιος 2024, θέμα 1 (α) — έλεγχος LL(1) · ⚙ (`2024-06.solutions.md`, χωρίς συμπέρασμα)
- Ιούνιος 2023, Παραλλαγή Α, θέμα 1 (β) — έλεγχος LL(1) · ⚙ (`2023-06-a.solutions.md`)
- Ιούνιος 2023, Παραλλαγή Β, θέμα 1 (β) — έλεγχος LL(1) · ⚙ (`2023-06-b.solutions.md`)
- Ιούνιος 2025, Παραλλαγή Α, θέμα 1 (α)–(δ) — τύπος, γλώσσα, έλεγχος `[][()]`, LL(1) · ⚙ (`2025-06-a.solutions.m` στο κεφάλαιο για το (δ))
- Ιούνιος 2025, Παραλλαγή Β, θέμα 1 — LL(1) και ανάλυση συμβολοσειράς · ✓ (`2025-06-b.solutions.md`, κοινοτική λύση μόνο για το (β))
- Ιούνιος 2026, θέμα 1 (δ) — LL(1) και αριστερή παραγοντοποίηση · ⚙ (στο κεφάλαιο)

Λεξική ανάλυση (κεφ. 6)

- Σεπτέμβριος 2013, θέμα 2 (α) — κανονική έκφραση για συμβολοσειρές που αρχίζουν και τελειώνουν με το ίδιο γράμμα · ⚙ (το υπόλοιπο θέμα δεν έχει μεταγραφή)
- Ιούνιος 2021, θέμα 1 (a)–(c) — αυτόματο, πίνακας, ίχνος για `001000` · ⚙ (στο κεφάλαιο)
- Ιούνιος 2022, θέμα 1 (e) — DFA για `(aa|bb)+` · ⚙ (στο κεφάλαιο)
- Σεπτέμβριος 2022, θέμα 1 — δυαδικοί αριθμοί μήκους ≥ 5 που αρχίζουν με `01` και τελειώνουν με `10` · ⚙ (χαμηλή, λείπει η αρχή του θέματος)
- Σεπτέμβριος 2024, θέμα 1 (e) — DFA για τη δεύτερη γραμματική (`(0|1)*0`) · ⚙
- Ιούνιος 2024, θέμα 2 (α) — αυτόματο με περιορισμό στο 0 · ⚙ (`2024-06.solutions.md`, ελλιπείς μεταβάσεις)
- Ιούνιος 2023, Παραλλαγή Α, θέμα 1 (στ) — DFA · ⚙ (`2023-06-a.solutions.md`)
- Ιούνιος 2023, Παραλλαγή Β, θέμα 1 (στ) — DFA · ⚙ (`2023-06-b.solutions.md`)
- Ιούνιος 2025, Παραλλαγή Α, θέμα 2 (α)–(β) — αυτόματο για το πολύ τρία `1` · ⚙ (`2025-06-a.solutions.md`)
- Ιούνιος 2025, Παραλλαγή Β, θέμα 2 — ίδια εκφώνηση με την Α · ⚙ (`2025-06-a.solutions.md`)
- Ιούνιος 2026, θέμα 2 (α)–(β) — αυτόματο και κανονική έκφραση · ⚙ (στο κεφάλαιο για το (α))

Συναρτησιακός: αποτίμηση (κεφ. 7)

- Σεπτέμβριος 2024, θέμα 4 (α) — βρόχος `while` ως αναδρομή με `j^i` · ⚙ (μέτρια)
- Σεπτέμβριος 2024, θέμα 4 (β) — τιμές `if` και `cond` στη Scheme · ⚙
- Σεπτέμβριος 2024, θέμα 4 (δ) — λ-λογισμός με τρία ορίσματα · ⚙ (στο κεφάλαιο)
- Σεπτέμβριος 2024, θέμα 4 (ε) — `square (sum [1:10])`, πρόθυμη και οκνηρή · ⚙ (στο κεφάλαιο)
- Σεπτέμβριος 2023, θέμα 6 — `square (sum [1:10])` · ⚙ (`2023-09.solutions.md`, στο κεφάλαιο)
- Σεπτέμβριος 2022, θέμα 5 (α) — λ-λογισμός με λογικές τιμές · ⚙ (στο κεφάλαιο)
- Σεπτέμβριος 2022, θέμα 5 (β) — `square (sum [1:100])` με τρεις στρατηγικές · ⚙ (χαμηλή στα βήματα)
- Ιούνιος 2022, θέμα 5 — βρόχος ως αναδρομή, λίστες, λ-λογισμός · ⚙ (χαμηλή, μερικό κείμενο)
- Ιούνιος 2023, Παραλλαγή Β, θέμα 4 (α) — `while` με `j^2i` σε αναδρομική μορφή · ○
- Ιούνιος 2025, Παραλλαγή Α, θέμα 6 (ε) — λ-λογισμός με τρία ορίσματα · ⚙ (στο κεφάλαιο)
- Ιούνιος 2026, θέμα 6 (Γ), ερώτημα (γ) — λογικές τιμές Church και `not` · ⚙ (μέτρια)
- Ιούνιος 2026, θέμα 6 (Δ) — `square(sum [1:100])` με τρεις στρατηγικές · ⚙ (χαμηλή στους αριθμούς)

Map, reduce και Prolog (κεφ. 8)

- Ιούνιος 2024, θέμα 6 (α) — άθροισμα κύβων με `map` · ⚙ (`2024-06.solutions.md`, στο κεφάλαιο)
- Ιούνιος 2024, θέμα 6 (β) — άθροισμα τετραγώνων με `reduce` · ⚙ (`2024-06.solutions.md`)
- Ιούνιος 2024, θέμα 6 (ε) — ζυγοί αριθμοί με `map` και `filter` · ○
- Ιούνιος 2025, Παραλλαγή Α, θέμα 6 (α)–(β) — `map` και `reduce` · ⚙ (`2025-06-a.solutions.md`, με προειδοποίηση για το (α))
- Ιούνιος 2025, Παραλλαγή Α, θέμα 6 (γ) — τι τυπώνει ένα `reduce` με γινόμενο · ⚙ (στο κεφάλαιο)
- Ιούνιος 2025, Παραλλαγή Α, θέμα 6 (στ) — SQL με `R1` και `R2` · ○
- Ιούνιος 2025, Παραλλαγή Β, θέμα 6 (α)–(β) — `map` και `reduce` για k-στές δυνάμεις · ○
- Ιούνιος 2026, θέμα 6 (Α) — Prolog: δέντρα, ανάγνωση κανόνα, ερωτήματα · ⚙ (στο κεφάλαιο)
- Ιούνιος 2026, θέμα 6 (Β) — SQL με `C < 400` και βελτιστοποίηση · ⚙ (μέτρια)
- Ιούνιος 2026, θέμα 6 (Γ), (α)–(β) — Python `map` και `reduce` · ⚙ (στο κεφάλαιο για το (β))
- Σεπτέμβριος 2024, θέμα 4 (γ) — `map` και `fold` στη Lisp · ⚙ (στο κεφάλαιο)

Σημείωση

Προσομοίωση. Διάλεξε θέματα για 2,5 ώρες (όσο διαρκεί η εξέταση) και γράψε όλα τα ενδιάμεσα βήματα. Για τα ίχνη (δείκτες, παράμετροι, στοίβα) μην κοιτάς τα παραδείγματα πριν λύσεις μόνος σου.

Πηγές και ευχαριστίες

- **Θέματα:** οι διδάσκοντες του μαθήματος. Παρατίθενται αυτούσια για σκοπούς μελέτης· τα δικαιώματα ανήκουν στους δημιουργούς τους.
- **Μεταγραφές:** η βάση γνώσης CEID, 12 εξετάσεις (2020–2026). Οι Ιούνιος 2021, Σεπτέμβριος 2022, Ιούνιος 2022, Ιούνιος 2023 (Παραλλαγή Α), Σεπτέμβριος 2023 και Ιούνιος 2026 είναι μερικές· όπου μια μεταγραφή είναι μερική, συμβουλέψου το πρωτότυπο.
- **Λύσεις:** τα αρχεία λύσεων της βάσης (Ιούνιος 2023 Α και Β, Σεπτέμβριος 2023, Ιούνιος 2024, Ιούνιος 2025 Α). Όλα είναι AI-unverified, εκτός από τη λύση «κοινοτική» του Ιουνίου 2025 Β (μόνο για το θέμα 1, με προειδοποιήσεις). Οι λύσεις που γράφονται εδώ ελέγχθηκαν με το χέρι και σημειώνονται AI.
- **Λύσεις AI:** για όσες εξετάσεις δεν υπάρχουν λύσεις, γράφτηκαν με τη βοήθεια AI και δεν έχουν ελεγχθεί από άνθρωπο· κάθε μία σημαίνεται.
- **Περίγραμμα και κατάλογος:** κατάλογος μαθημάτων 2025-26 (ECTS, ώρες, διδάσκοντες).
- **Διαφάνειες και ασκήσεις eClass:** `eclass/Chapter_7.pdf.md` (υποπρογράμματα και τρόποι μεταβίβασης παραμέτρων), `eclass/16.Functional - Programming (Στρατηγικές Αποτίμησης).pdf.md` (πρόθυμη και οκνηρή αποτίμηση, αναγωγές).
- **Γνώμες φοιτητών:** το κανάλι του μαθήματος στο Discord του CEID, ανώνυμα και με ημερομηνία, όπως καταγράφονται στο README του μαθήματος.

Κείμενο, επεξηγήσεις και έλεγχος λύσεων: Giorgos Kandilas, με τη βοήθεια AI.

Βρήκες λάθος ή έχεις επίσημες λύσεις ή θέματα που λείπουν (π.χ. Σεπτέμβριος 2025, Ιανουάριος 2026)· στείλ' τα· κάθε συνεισφορά αναφέρεται στην επόμενη έκδοση.

Πώς να συνεισφέρεις: διορθώσεις, θέματα που λείπουν και επίσημες λύσεις δέχονται στο `https://kdngiorgos.github.io/ceid-notes/` — κάθε συνεισφορά αναφέρεται στην επόμενη έκδοση.